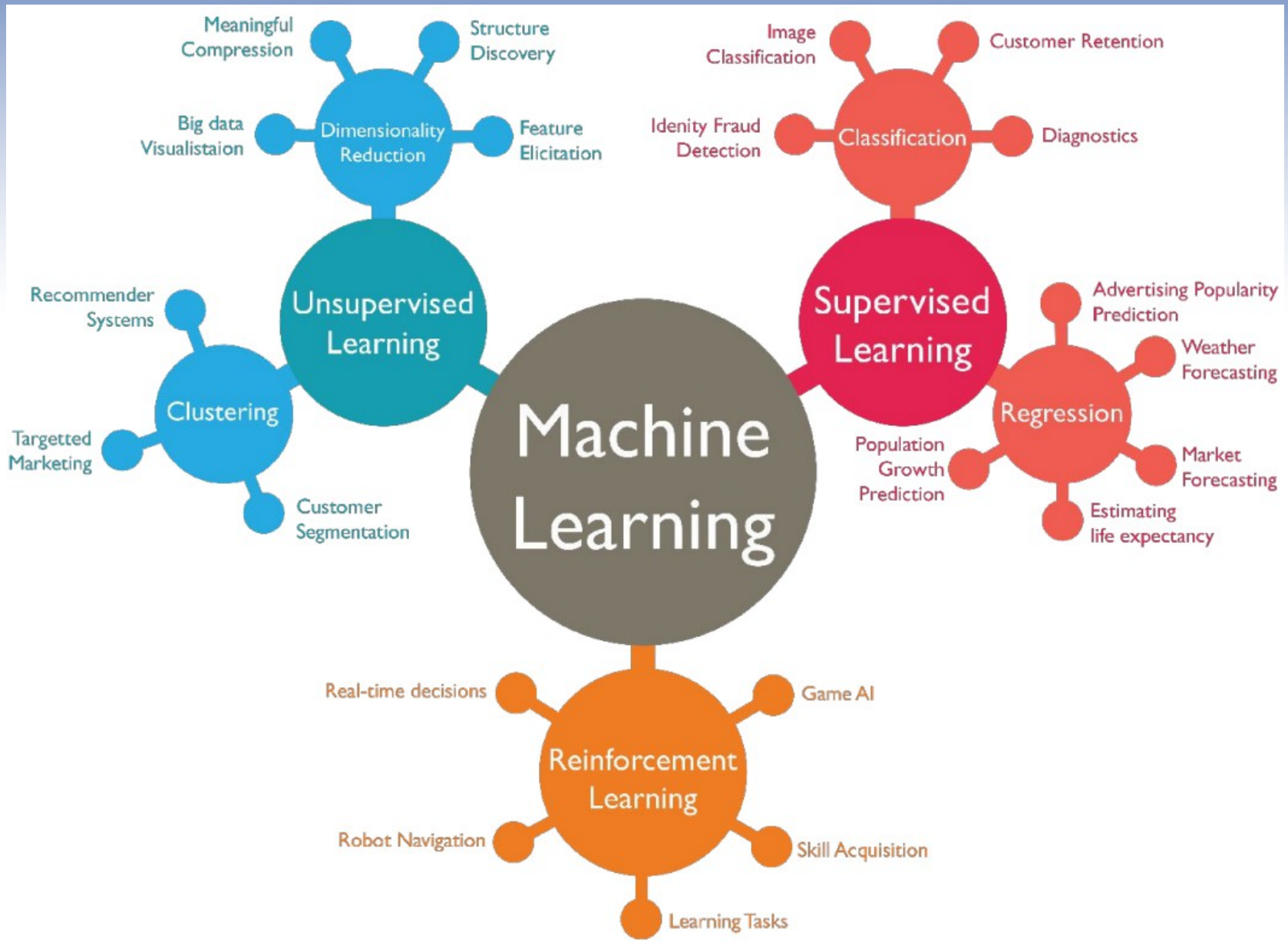

Race Vision

Why Machine Learning ?

A promising technology for a successful comeback

What is Machine Learning ?



What is Machine Learning ?

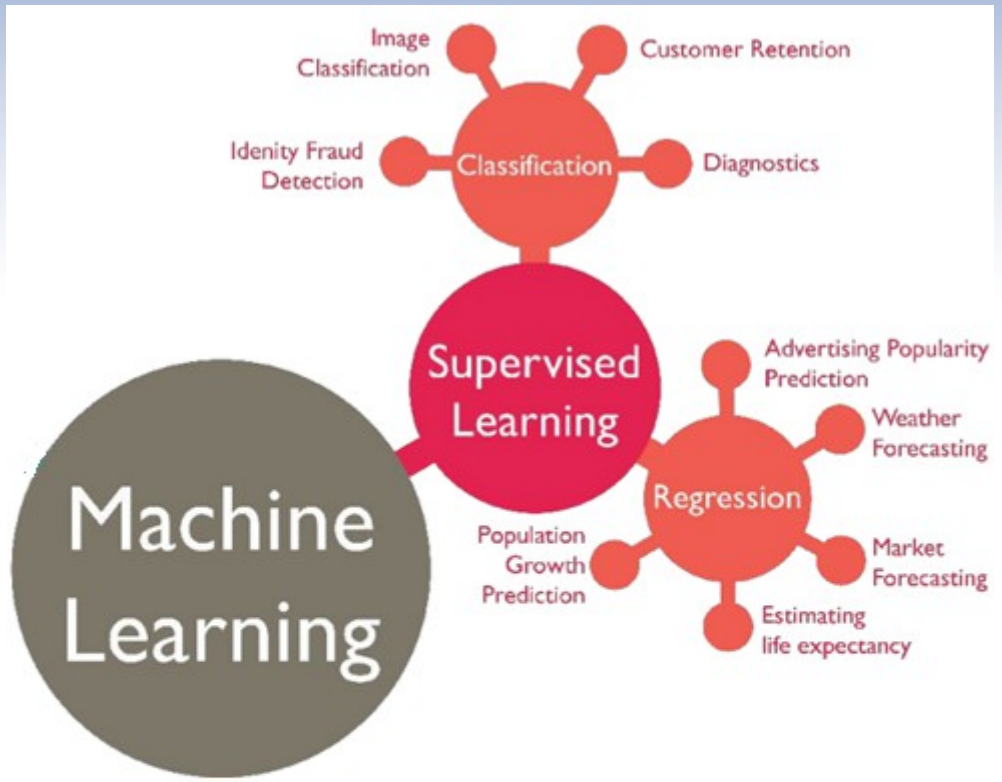
Supervised Learning

- The algorithm is trained on a labeled dataset
We give it both input and expected output

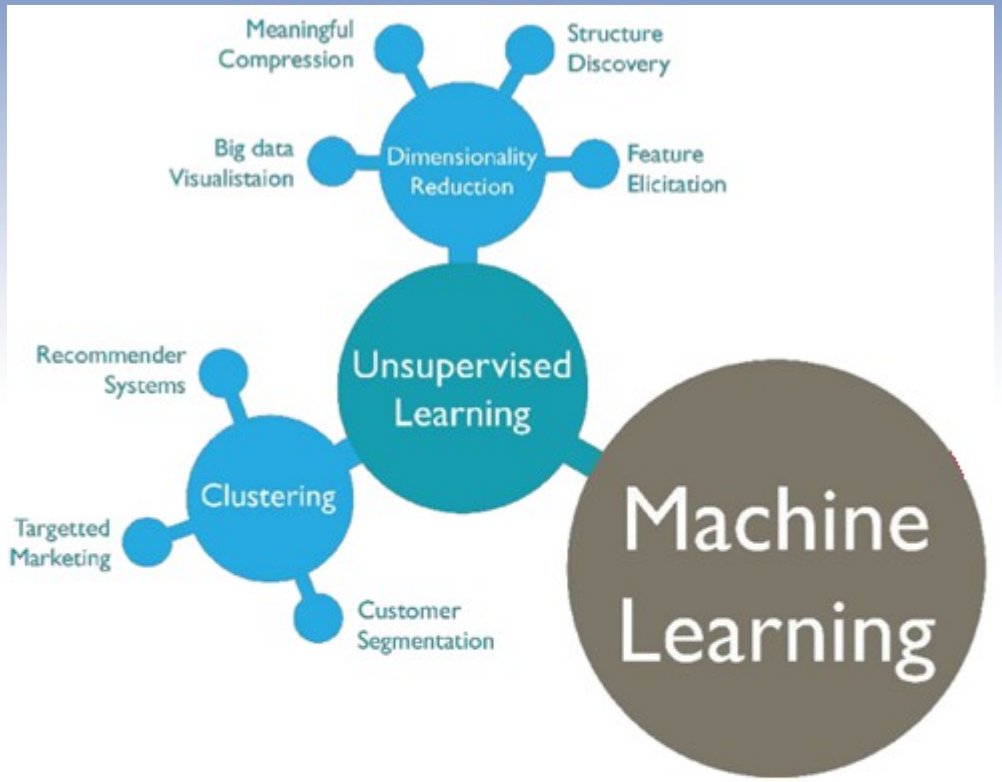
Tasks

- Classification: Predict a discrete category.
→ Email spam detection (spam vs. Not-spam)

- Regression: Predict a continuous value.
→ House-price estimation based on known size, location, etc.



What is Machine Learning ?



Unupervised Learning

- Works with unlabeled data
- The model tries to discover structure or patterns without predefined labels.

Tasks

- Clustering: Group similar points together.
→ Customer segmentation in marketing.

- Dimensionality Reduction: Find a lower-dimensional representation that preserves important structure.
→ Principal Component Analysis (PCA) for visualizing high-dimensional data.

- Association Mining: Discover rules that describe large portions of data.
→ Market-basket analysis (“if customers buy bread, they often buy butter”).

What is Machine Learning ?

Reinforcement Learning

- Based on an agent interacting with an environment recieving (or not) rewards.
This way, it defines the best selection of actions.

Tasks

- Control & Navigation

→ Guiding a robotic arm to grasp objects

→ Planning paths for an autonomous vehicle.

- Game Playing

→ Learning to play games.

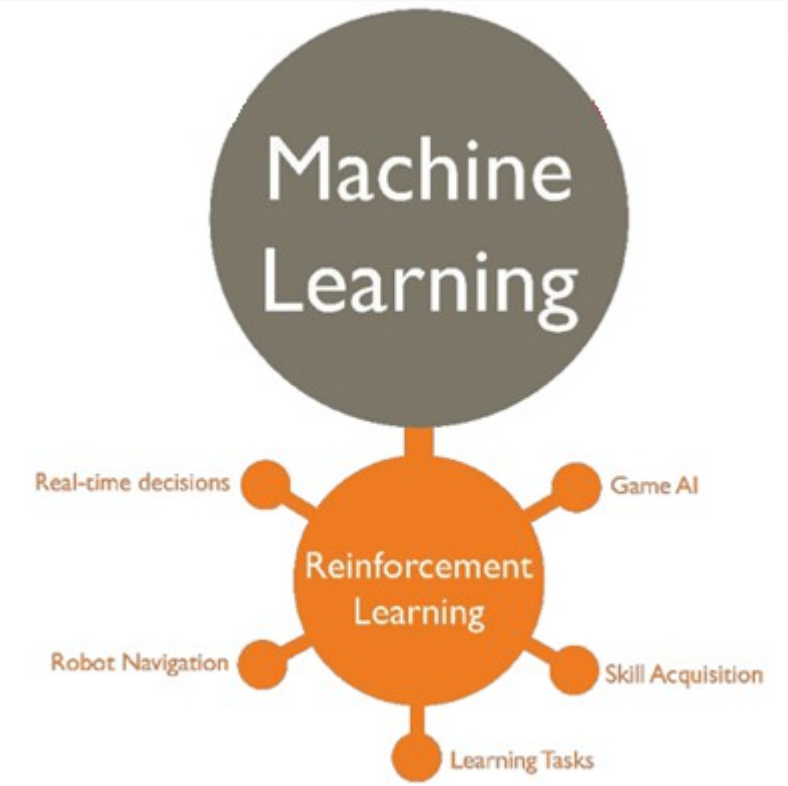
- Resource Management

→ Dynamically allocating servers in a data center to minimize energy use.

- Dialogue & Recommendation Systems

→ A chatbot that chooses keeping a user engaged

→ A recommender that sequences content to maximize long-term user satisfaction.

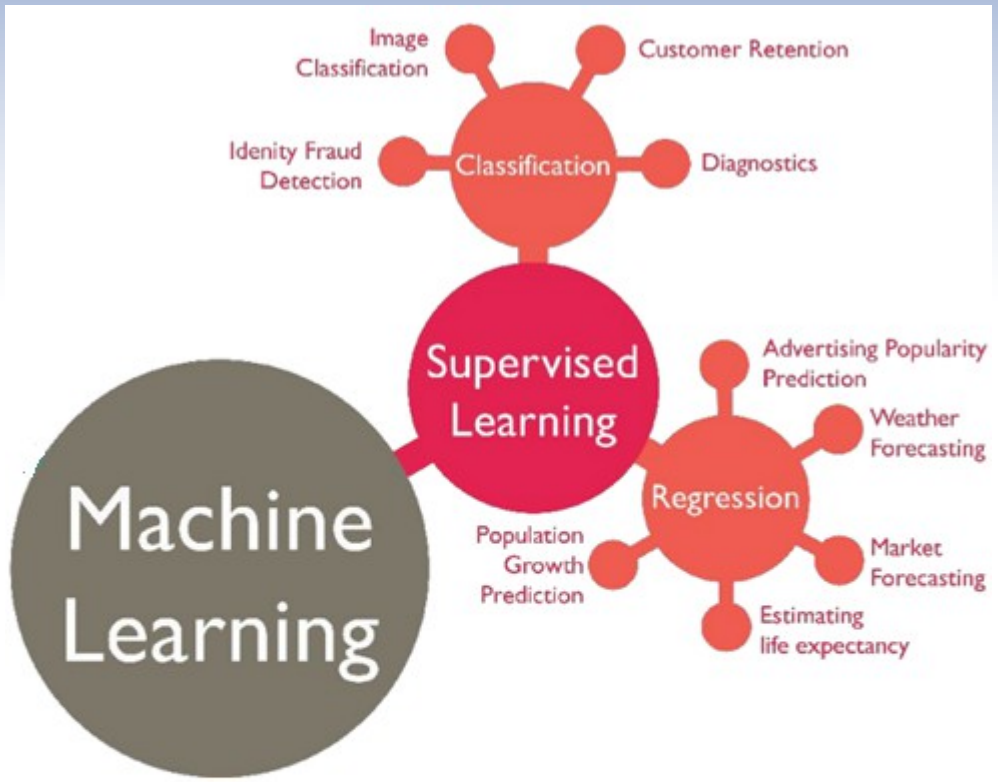


Regarding this project

Supervised Learning

Input / Train :
Set of previous races results (2010-2023)

Test :
- Try to predict The 2024 results
- Compare it with the know results



Back to the project

Description

RaceVision is a deep learning pipeline for lap-by-lap prediction of Formula 1 races.

→ Uses **historical race data** from the Ergast Motor Racing Database

→ Builds time-series **datasets**

→ **Trains** an LSTM model to forecast for each driver :
- Actual positions
- Lap times
- Pit-stop events
- Status (racing or type of incident)

Files structure

```
RaceVision/
├── db/                                  # Processed data directory (user-specified)
│   ├── races/                           # Generated per-race CSVs by script 1
│   │   └── <year>/race<raceId>.csv
│   ├── drivers_short.csv                # Unique driverId listing
│   └── races_npy/                       # Generated .npy inputs/outputs by script 2
│       └── <year>/{idx}_in.npy
│                  {idx}_exp.npy
├── ergast/                              # Raw data directory (data from Ergast DB)
│   ├── races.csv                        # Metadata: race calendar
│   ├── lap_times.csv                    # Raw lap-time logs
│   ├── results.csv                      # Race results per driver
│   ├── qualifying.csv                   # Qualifying session times
│   ├── pit_stops.csv                    # Pit-stop events (from 2012)
│   ├── driver_standings.csv             # Championship standings per round
│   └── constructor_standings.csv        # Constructor standings per round
├── notebooks/
│   ├── 1-DataPreparation.py             # Converts raw Ergast DB CSVs to per-race/lap CSVs
│   ├── 2-RacePrediction_GRU.py          # Builds embeddings, .npy datasets, and defines & trains GRU model
│   └── 2-RacePrediction_LSTM.py         # Builds embeddings, .npy datasets, and defines & trains LSTM model
├── presentation/                        # This presentation
│   └──  images/                         # Images used for the presentation
├── sd/                                  # Model checkpoints and final weights
├── testing/                             # Metrics of the tests done
└── README.md                            # (this file)
```

Data gathreing

Source

Our data are from Ergast DB : http://ergast.com/mrd/db/

Ergast files used :

- races.csv : race metadata (year, round, circuit, etc.)
- driver_standings.csv : standings after each round
- constructor_standings.csv : standings after each round
- qualifying.csv : Q1/Q2/Q3 lap times
- pit_stops.csv : pit stop events (from 2012 onward)
- lap_times.csv : every driver's lap-by-lap time
- results.csv : final finishing order and status per driver per race

Data preparation

File structure

The **input files** contains 1 row per race lap.

→ Each row contains all the datas regarding the race and each driver

- Lap 0 is the pre-race "qualifying" snapshot

Dataframe Structure		
circuitID	ID of the circuit where the race is held	integer
For each of the 20 grid slots (k = 1…20), seven fields:		
driverId{k}	The driver's unique ID	integer
driverStanding{k}	That driver's championship standing before this race	integer
constructorStanding{k}	That driver's constructor's standing before this race	integer
position{k}	Driver's track position at the end of this lap (1…20; 21 = retired…)	integer
inPit{k}	1 if the driver pitted on this lap; otherwise 0	integer
status{k}	Status code (0 = racing, non-zero = retirement/other)	integer
laptime{k}	Lap time in seconds (or qualifying time on lap 0; 0 if missing/retired)	float or 0

One-hot

Data normalization

Numeric value	0	1	2	3	4	5	6	7	8	9	10
One-hot	0	0	0	0	0	0	1	0	0	0	0

Advantages

1 - Ordinality

Integers imply order :
driverId 10 may be understood as “twice” driverId 5 or “more” in some monotonic sense.

2 - Sparse, High-Dimensional Representations

Sparsity (mostly zeros, one “1”) leads to more stable gradients and less interference between categories.

3 - Consistent Scaling

Mixing integers of wildly different ranges (e.g. driverId up to 130, lap time ~100 s, positions 1–21) may be tricky to normalize.
One-hot entirely sidesteps that, since all features are in {0,1}.

In practice for my script

- Circuit ID, driver ID and position are all categorical.

- By embedding each as a one-hot of length 130, 130, or 21 respectively, I give my LSTM a clean, non-ordinal signal.

- The neural network can then learn -for example- that driver 7 and driver 12 have entirely different learned interactions with lap percentage or pit flags, without confusing them because of a numeric scale.

Data setup

.npy input

Here is the input file after « normalization »

Dataframe Structure		
circuitID	ID of the circuit where the race is held	One-hot 130d
Lap number	lap in race percentage progress	Float
For each of the 20 grid slots (k = 1…20), seven fields:		
driverId{k}	The driver's unique ID	One-hot 130d
driverStanding{k}	That driver's championship standing before this race	One-hot 3d
constructorStanding{k}	That driver's constructor's standing before this race	One-hot 2d
position{k}	Driver's track position at the end of this lap (1…20; 21 = retired…)	One-hot 21d
inPit{k}	1 if the driver pitted on this lap; otherwise 0	Binary
status{k}	Status code (0 = racing, non-zero = retirement/other)	One-hot 6d
laptime{k}	Lap time in seconds (or qualifying time on lap 0; 0 if missing/retired)	One-hot 32d

Data setup

Why ?

Question :

Why 130d for driversId and Circuit Id ?

Data setup

.npy output

Each row corresponds to the predicted position of each driver for lap i+1

→ Given inputs from lap i.

Dataframe Structure		
For each of the 20 grid slots (k = 1…20), seven fields:		
position{k}	Driver's track position at the end of this lap (1…20; 21 = retired…)	One-hot 21d
laps-to-pit{k}	1 if the driver pitted on this lap; otherwise 0	Binary
status{k}	Status code (0 = racing, non-zero = retirement/other)	One-hot 6d
laptime{k}	Lap time in seconds (or qualifying time on lap 0; 0 if missing/retired)	One-hot 32d

Model choice

What we expect from our model :

- Remember what happened during previous laps

→ Races can be 50+ laps long, and key events (pit-stops, tyre-degradation) often happen many laps earlier.

→ The driver leading the championship may not suddenly become slow because of an accident.

- Length of variables may not be fix

→ Different Grands Prix have different numbers of laps

- Manage the volume of datas

→ 20 drivers × ~50 laps × ~20 races × ~20 years

- Different types of variables

→ We use one-hots, scalars and binaries

- Easy to handle

→ Time saving is key to respect the deadline

Why LSTM Model

What LSTM does :

- Sequential Memory

→ LSTM's gated cell state lets it learn to store and forget information as needed, preserving long-range dependencies over 50+ laps.

- Variable-Length Races

→ LSTMs naturally handle sequences of varying length without needing to re-architect the model: you just feed in as many time-steps as that race has.

- Data Volume

→ LSTMs with a few layers (e.g. 2×1200) give plenty of capacity without exploding your parameter count or requiring tens of millions of examples.

- Feature Heterogeneity

→ LSTMs accept flat real-valued vectors, so you can concatenate everything without special accommodations.

- Ease of Training & Tuning

→ LSTMs are a mature, well-understood primitive in PyTorch: there's standard support for truncated BPTT, built-in dropout between layers, Xavier init, etc.

Why LSTM Model

Other contenders :

- GRU

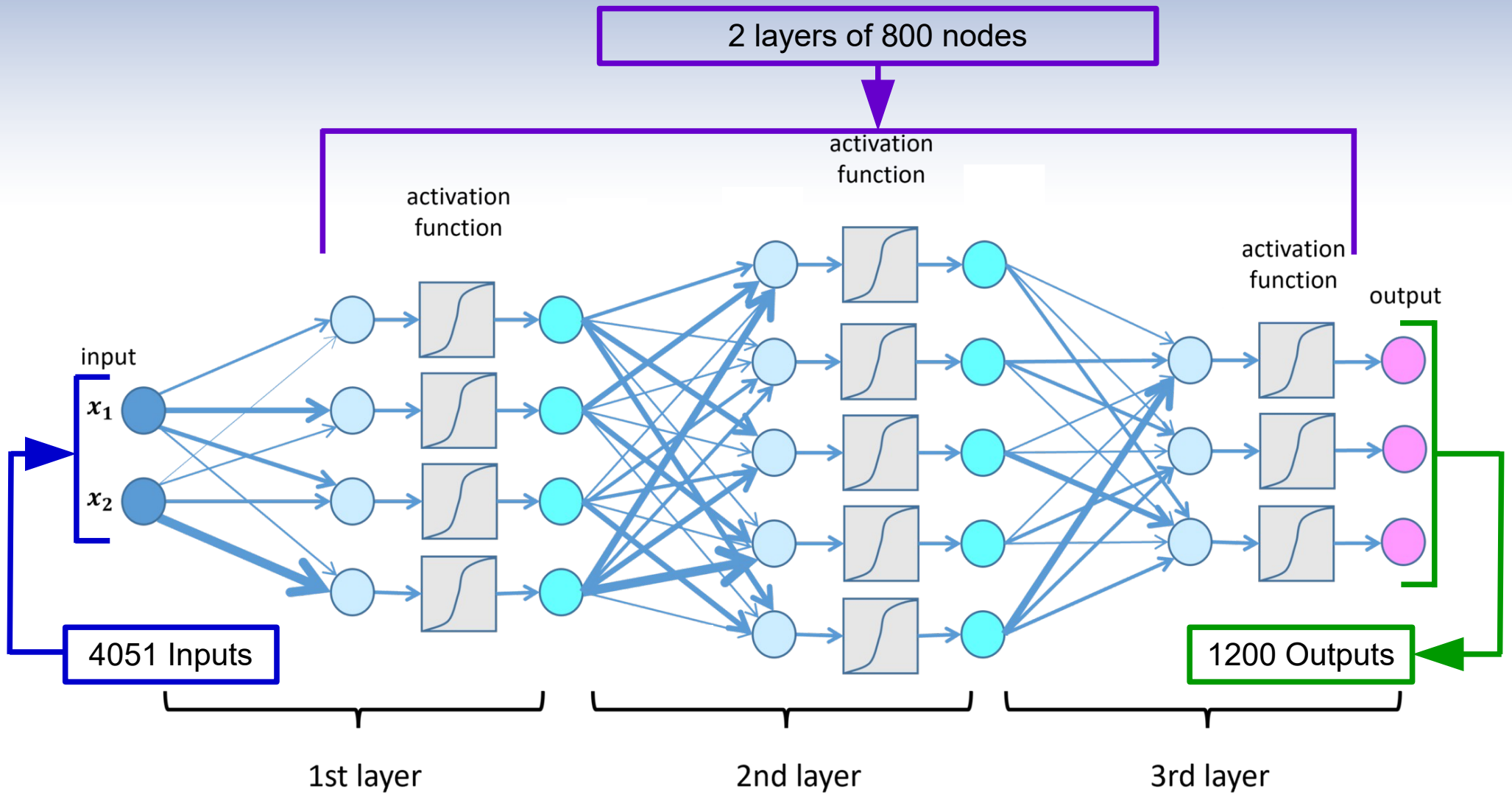
→ fewer parameters

- Temporal CNN

→ faster parallelizable convolutions

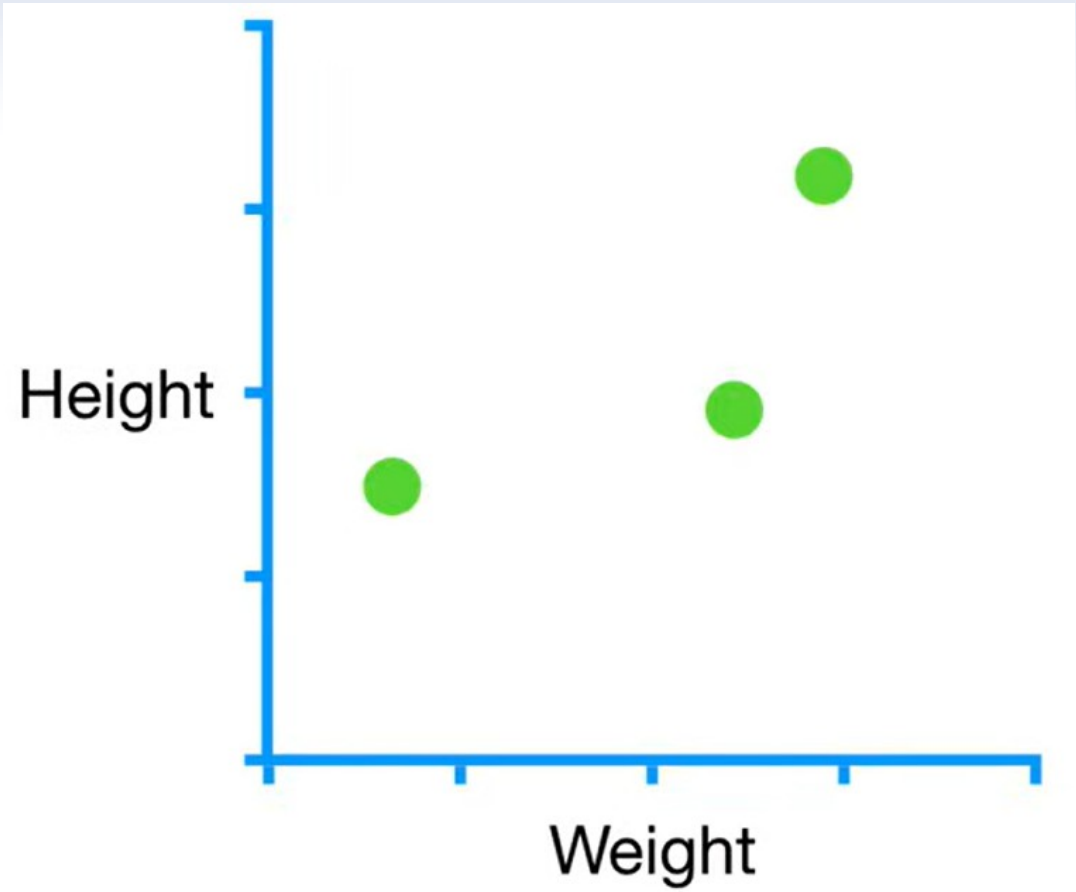
The Neural Network

Enter the black box



Optimization

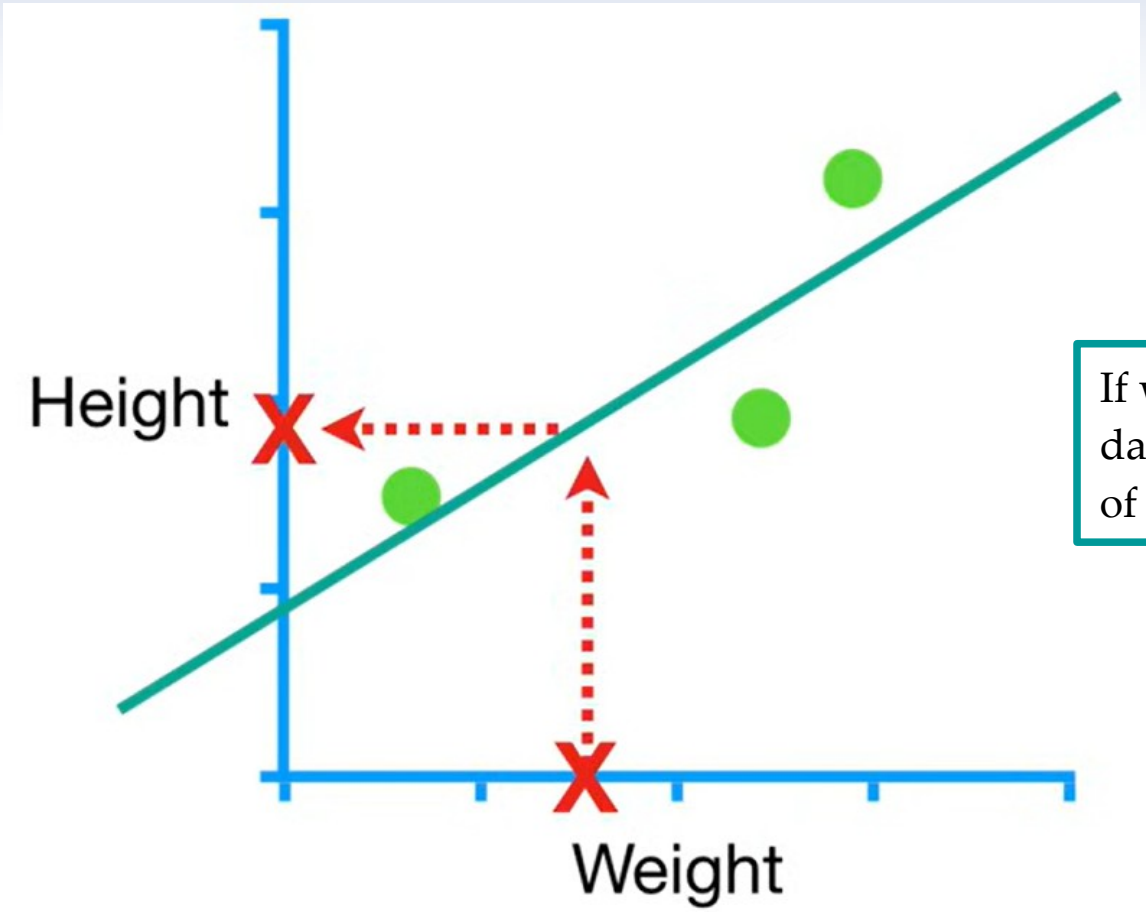
How the machine learns



Here is a dataset tracking the relationship between Weight and Height

Optimization

How the machine learns



If we draw a line following the trend of the dataset, me may be able to predict the Height of someone knowing its Weight

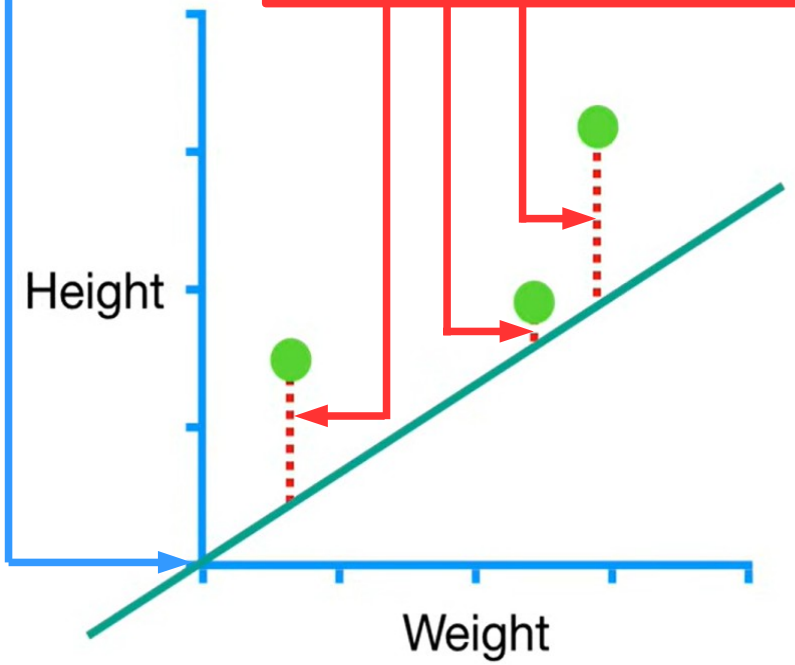
Optimization

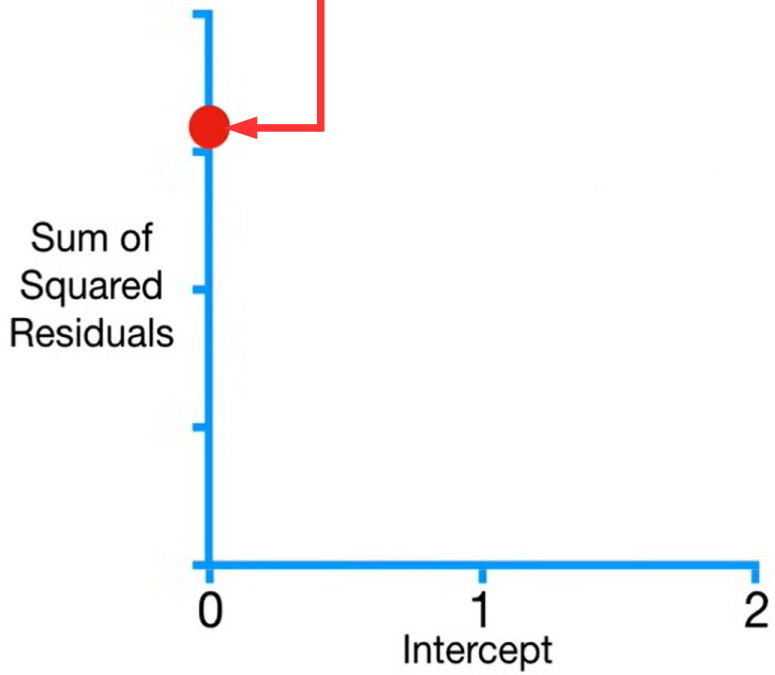
How the machine learns

1. To find the best y axis value for our line, westart at the 0 position

2. We sum the distance between the dots and the line

3. We plot that value on a graph

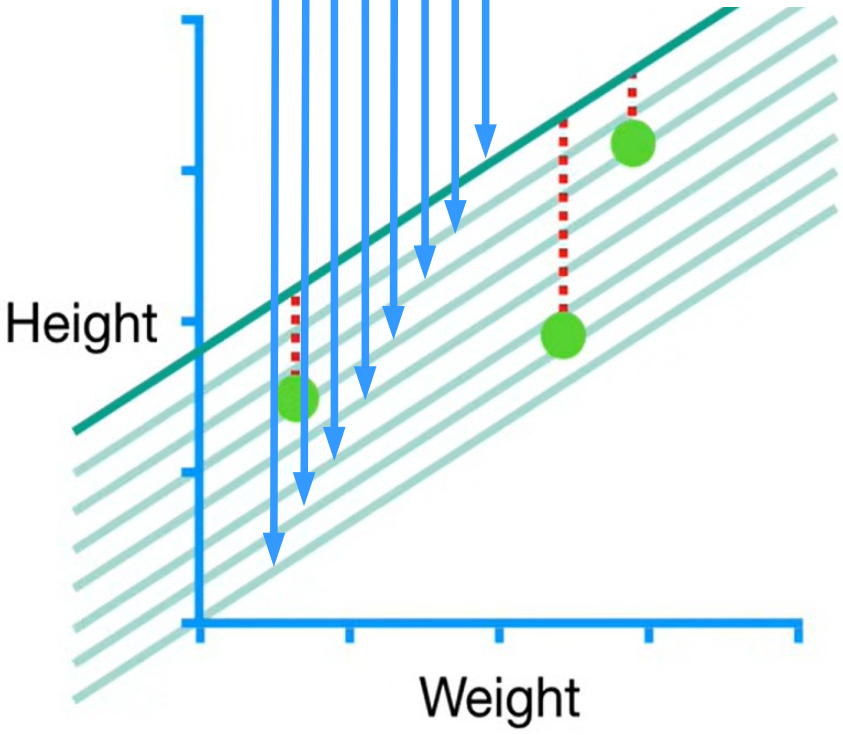


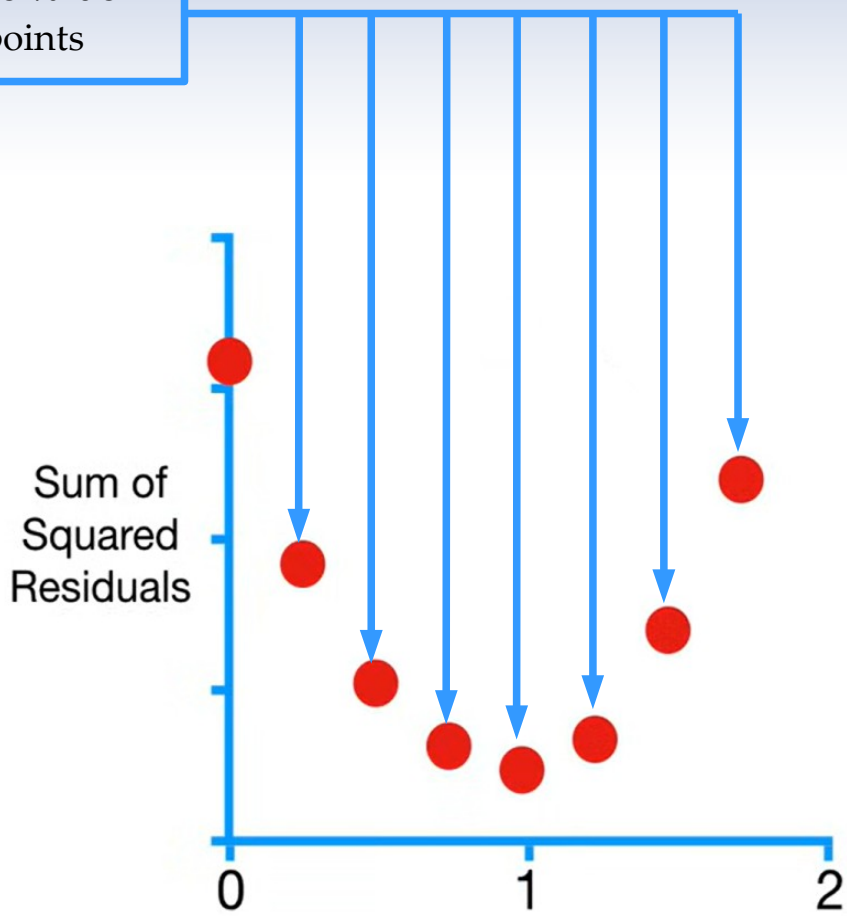


Optimization

How the machine learns

We progressively increase the value on the y axis, and get these points



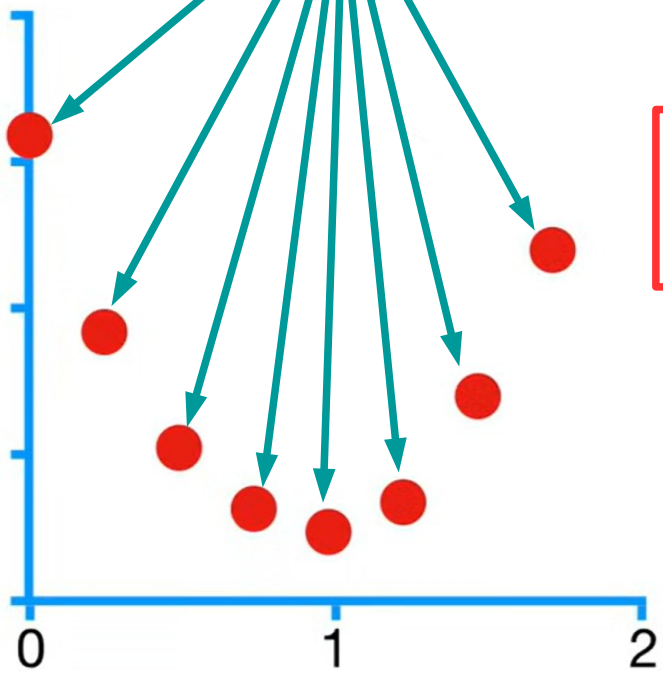


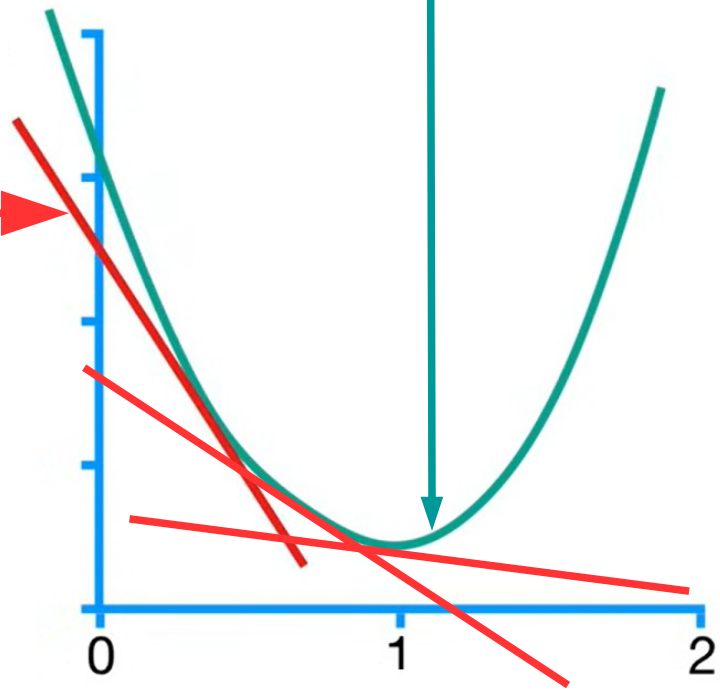
Optimization

How the machine learns

From these values, we define an equation

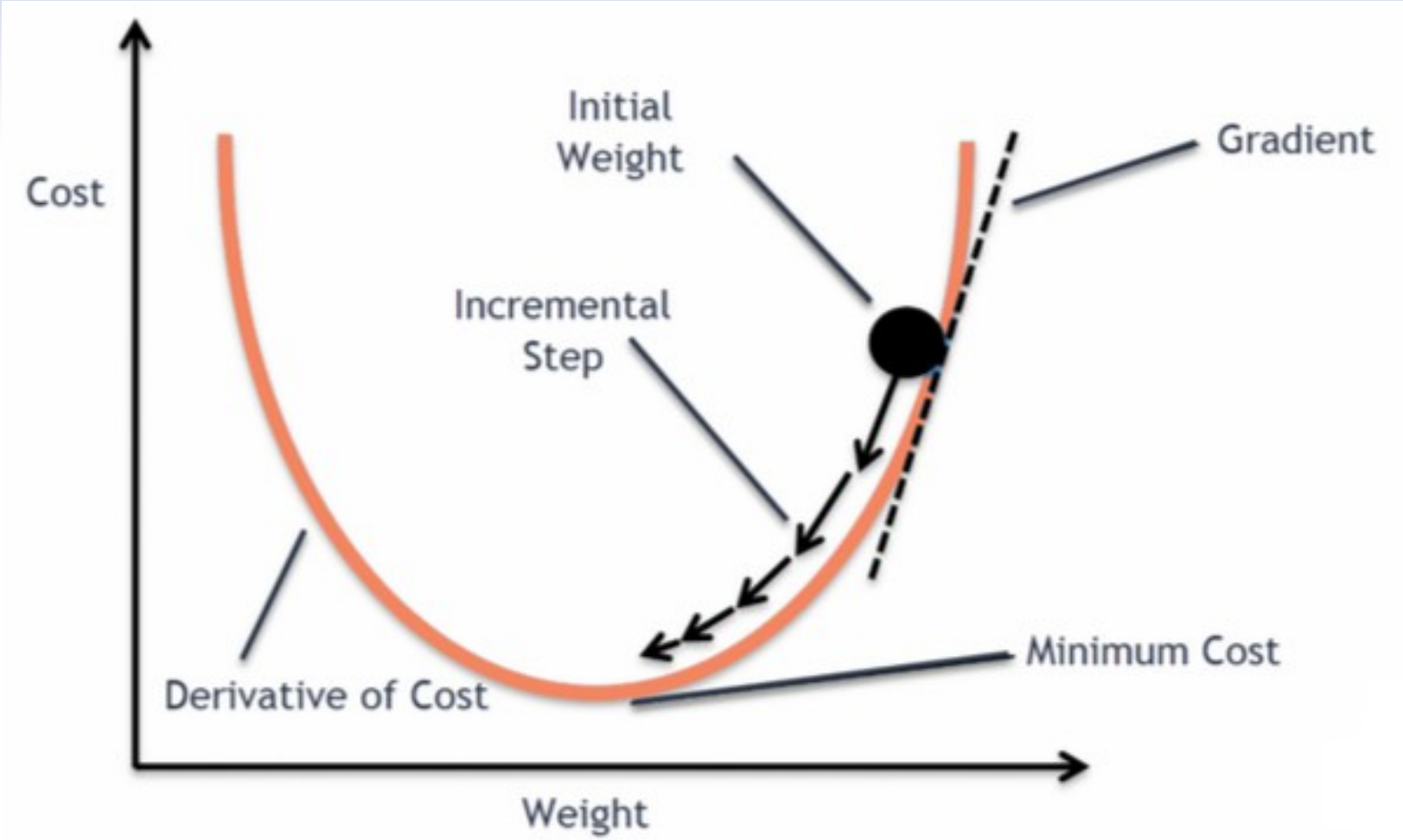
Using the derivative of this function, we determine the slope at any value





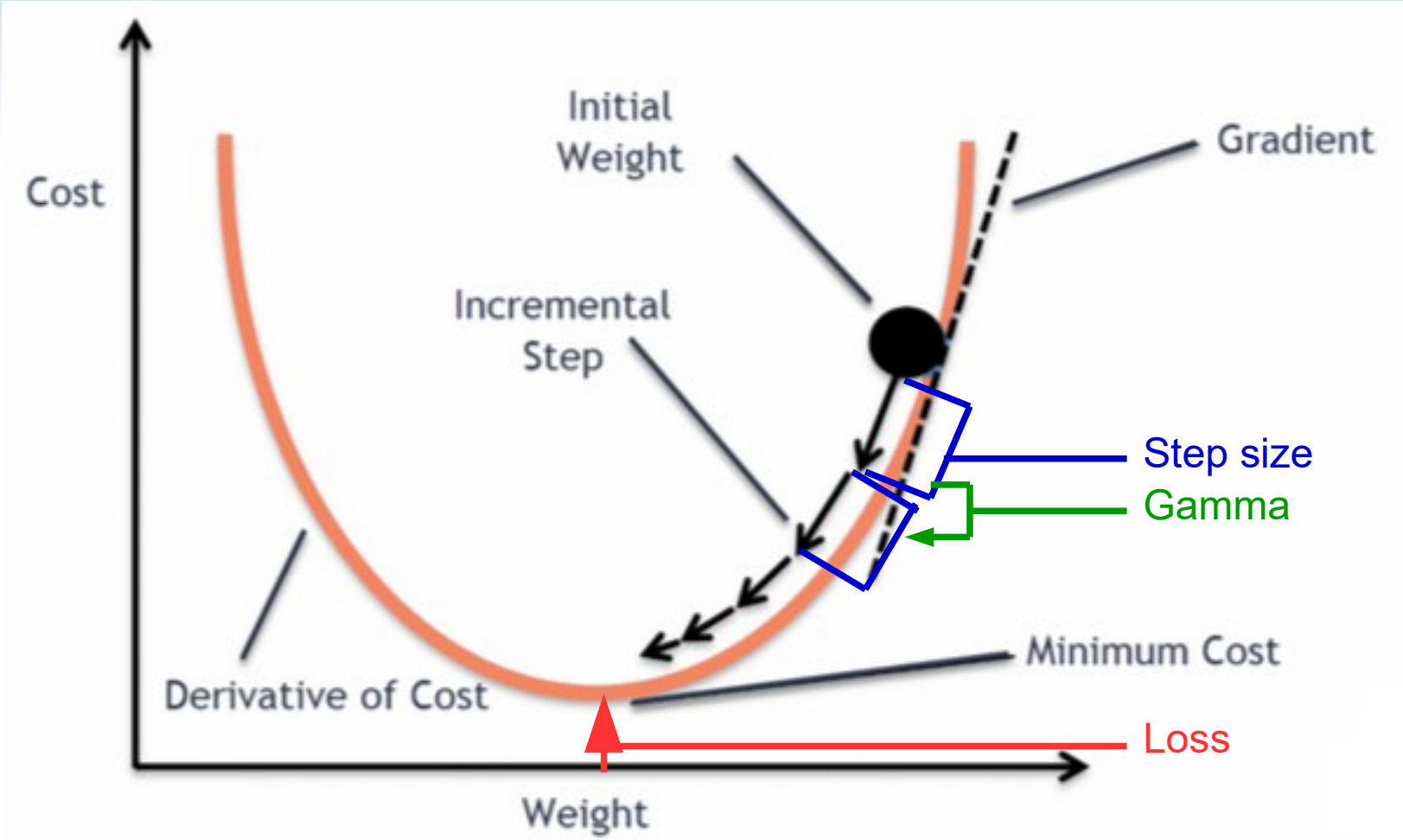
Optimization

The gradient descent



Optimization

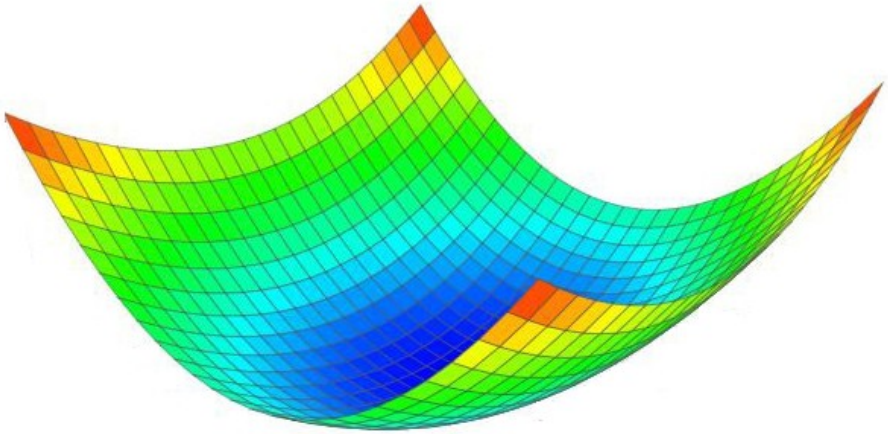
Tuning

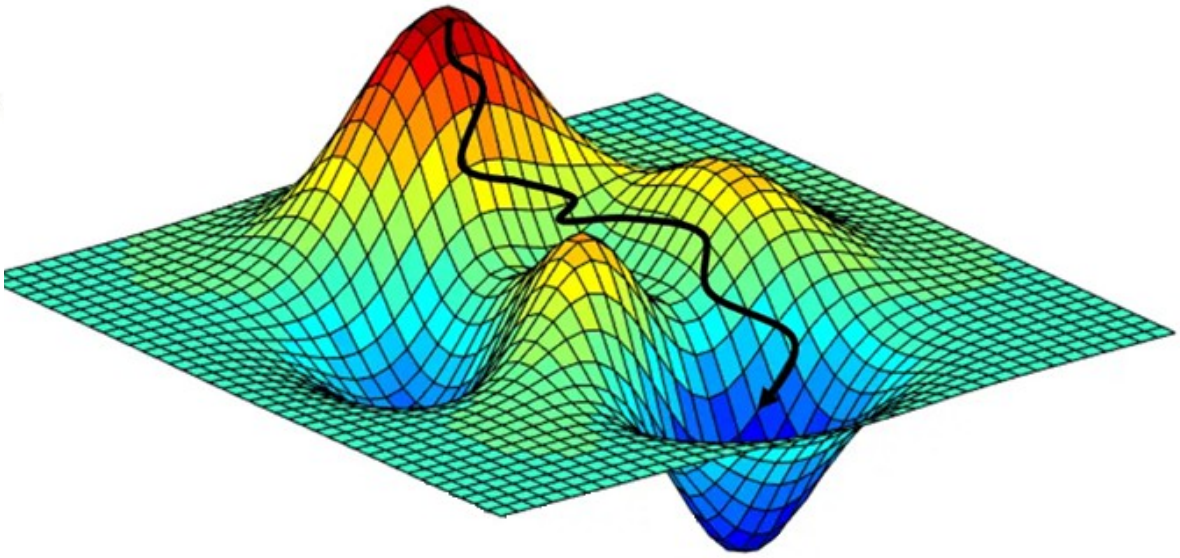


Optimization

To infinity and beyond !

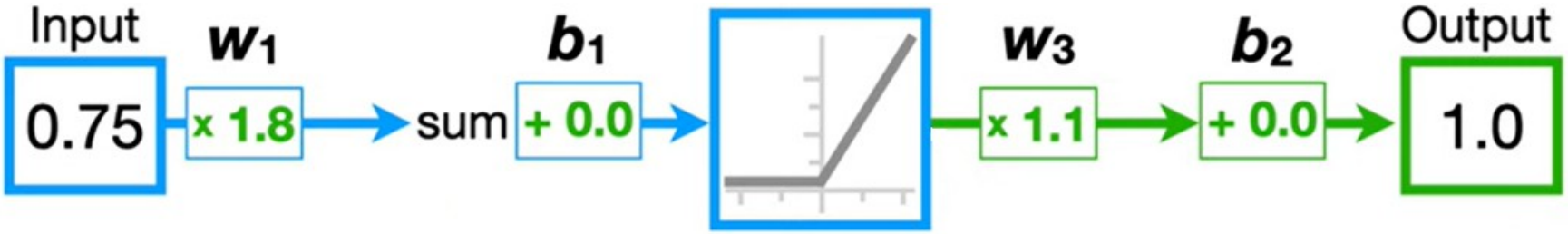
This was a very basic example
The gradient descent can be very tricky





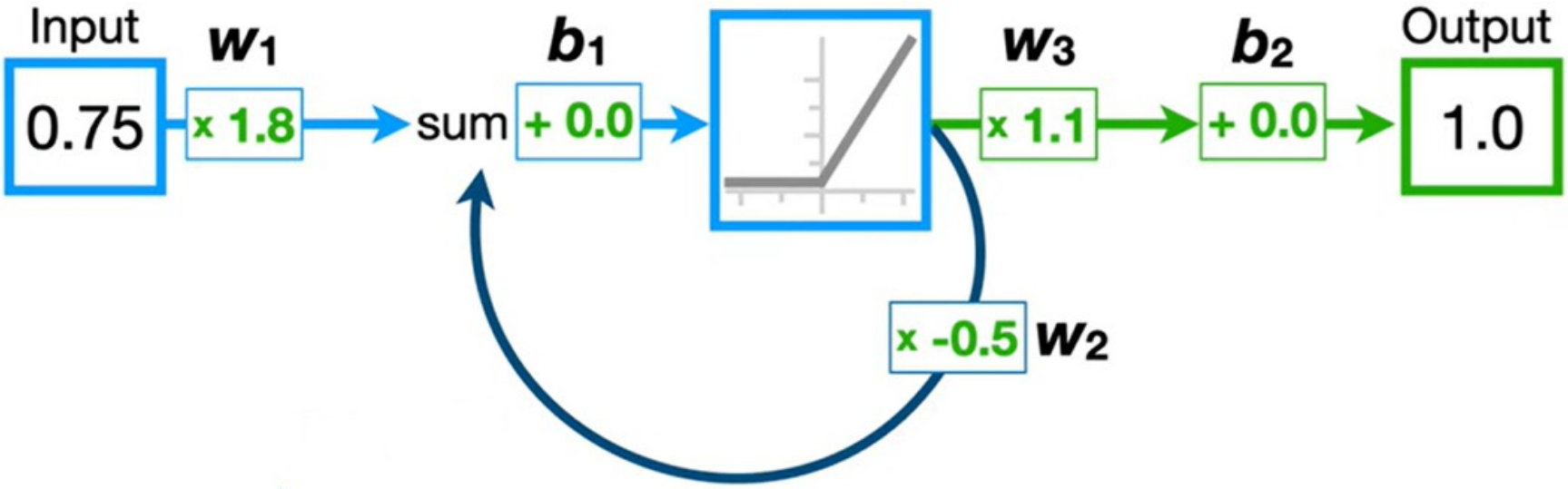
The Recurrent NN

Get simple



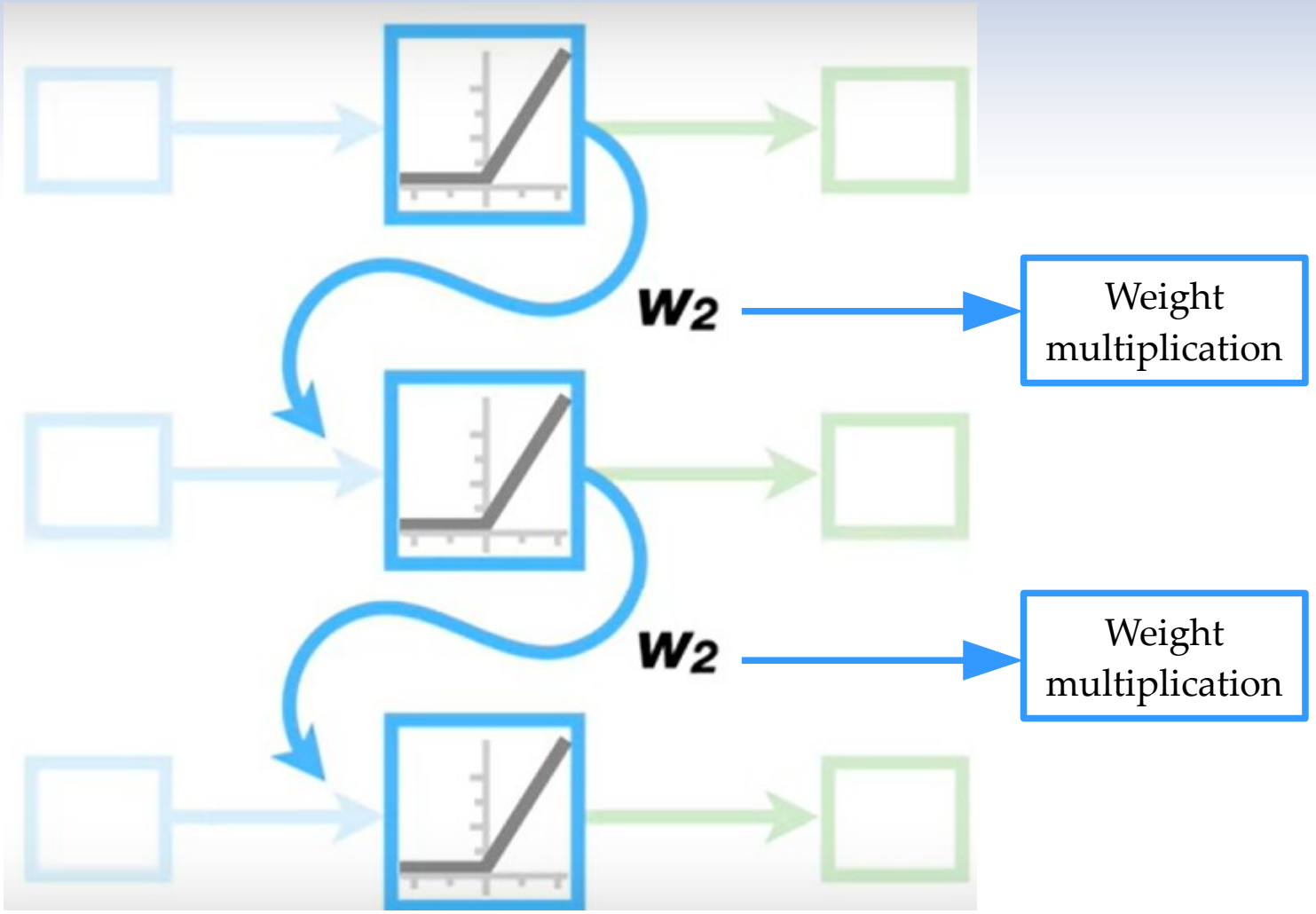
The Recurrent NN

Into the loop



The Recurrent NN

Unroll the loop

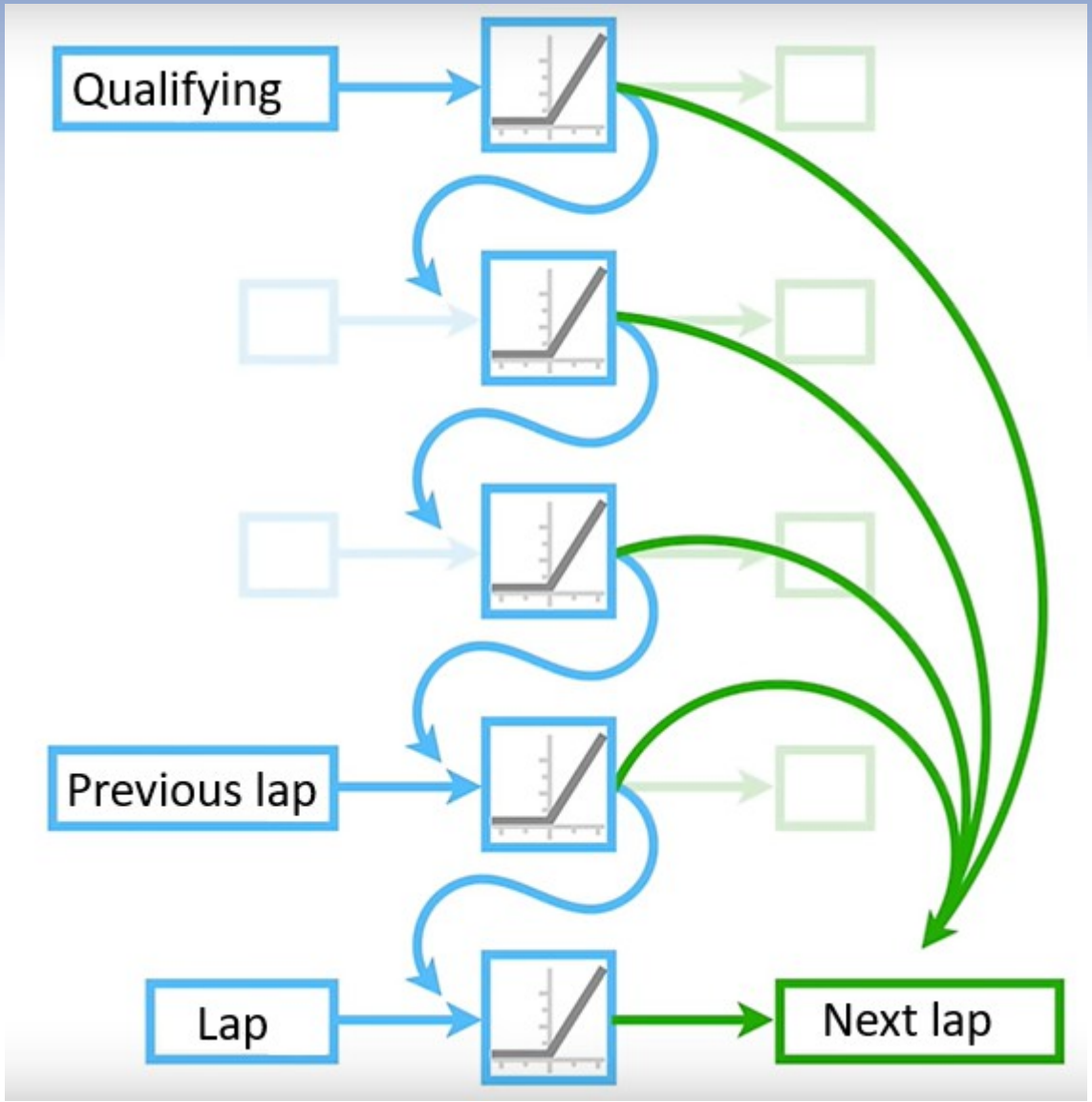


How to fix it ?


BWT
ALPINE
1° TEAM
RENAULT
E-TECH
MAPFRE
BWT
DUPONT
Microsoft
BINANCE
CHANGE THE WORLD
step by step
RACEFANS.NET

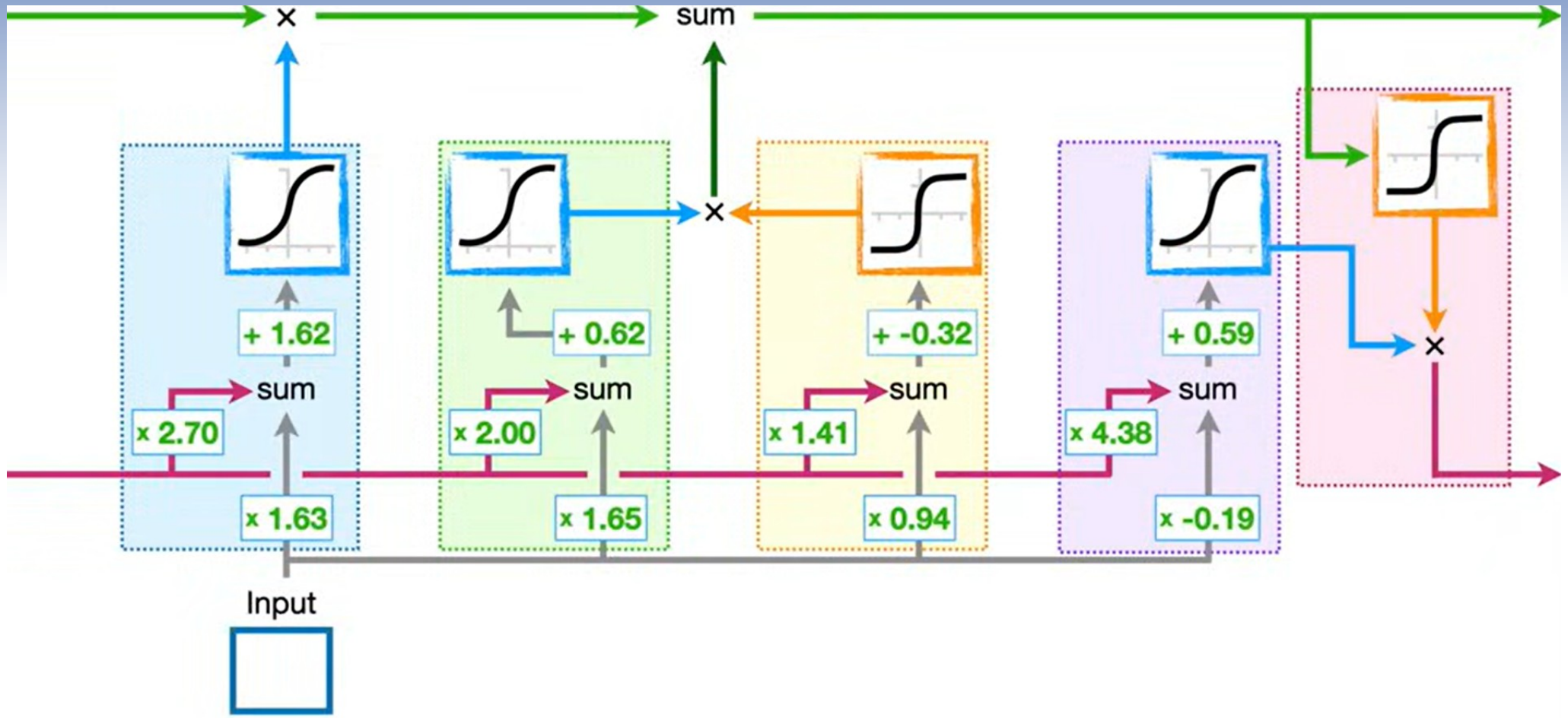
How LSTM works

The principle



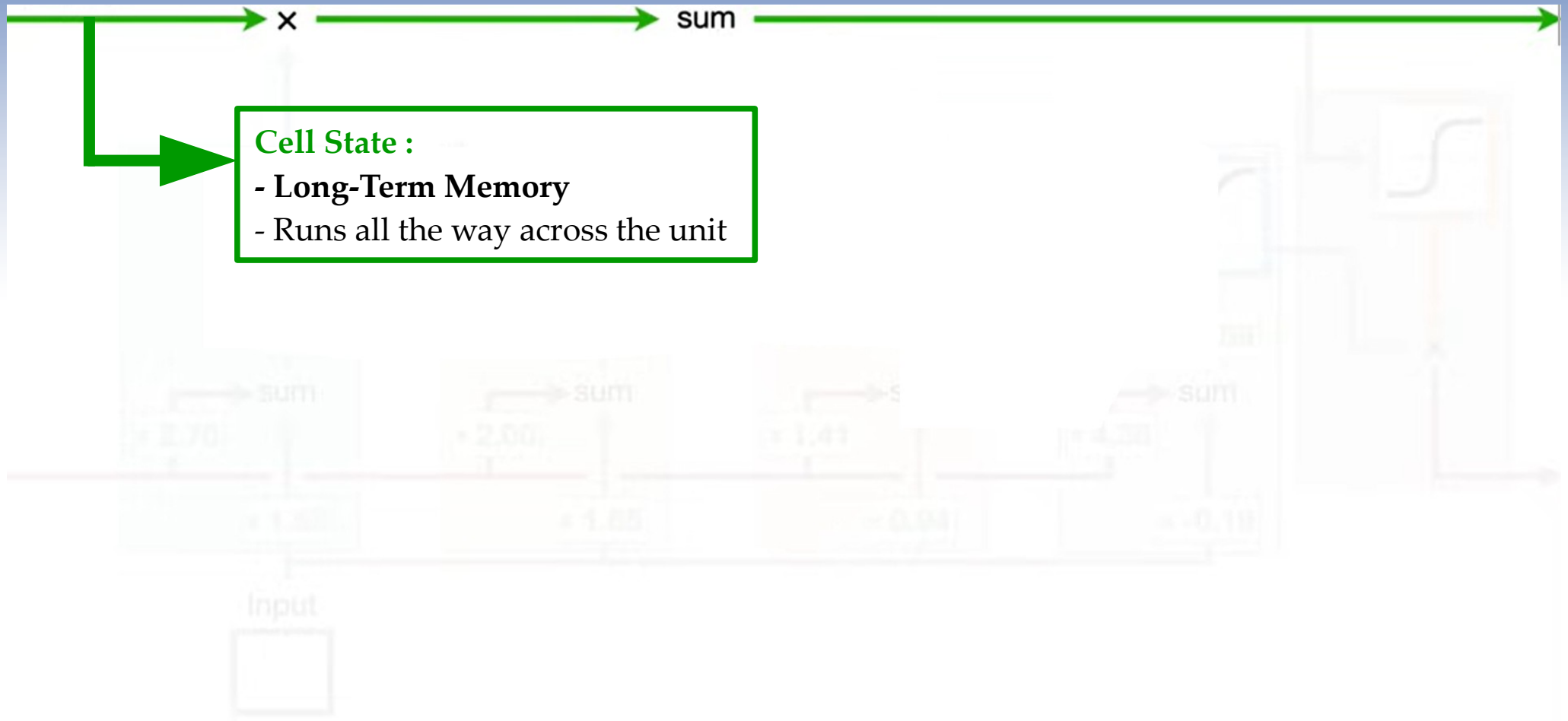
The LSTM unit

Enter the unit



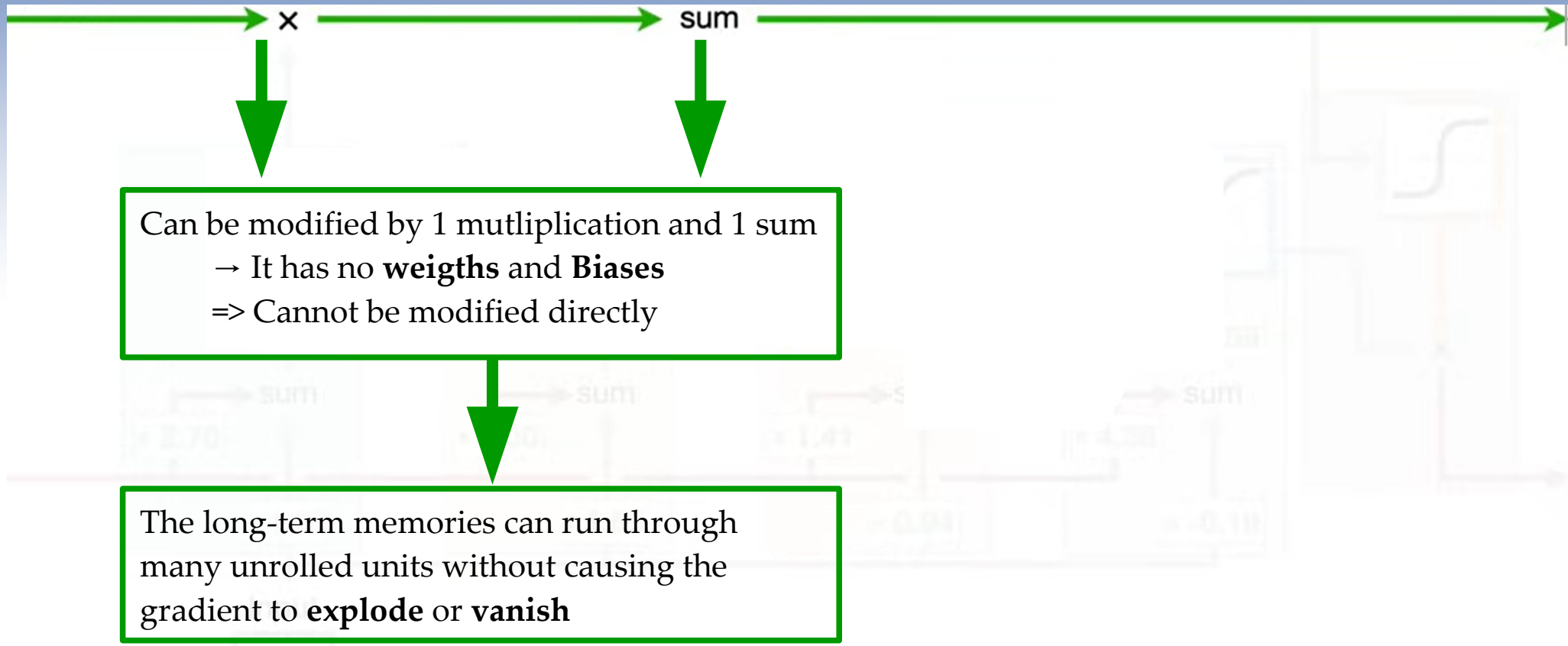
The LSTM unit

The Long-Term Memory



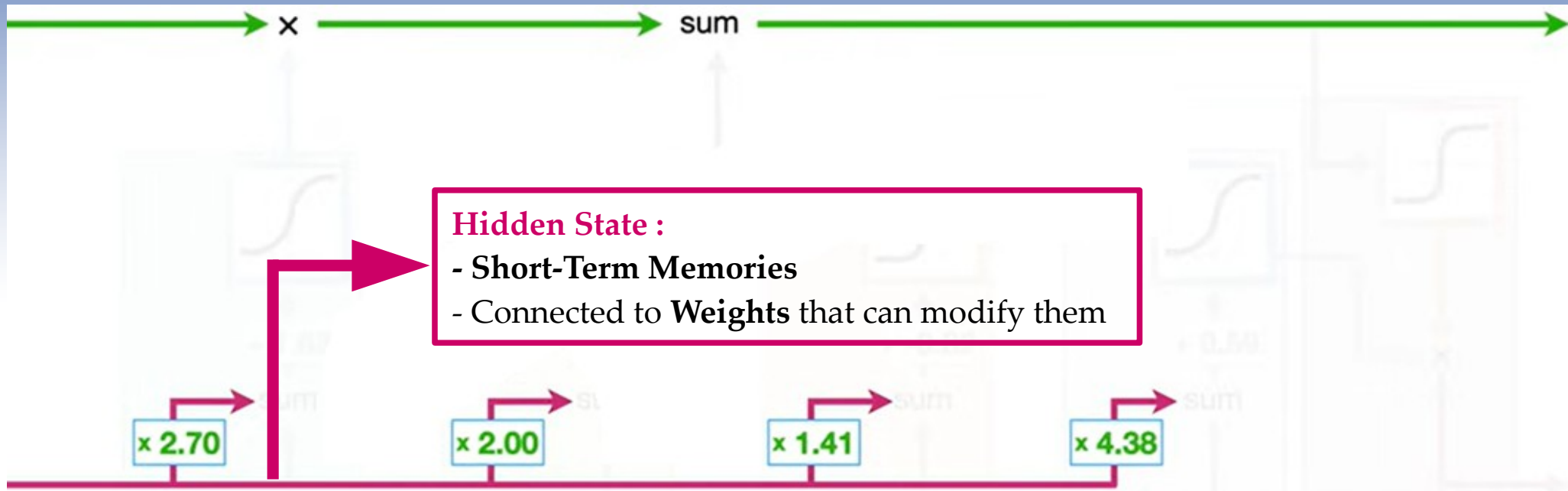
The LSTM unit

The Long-Term Memory



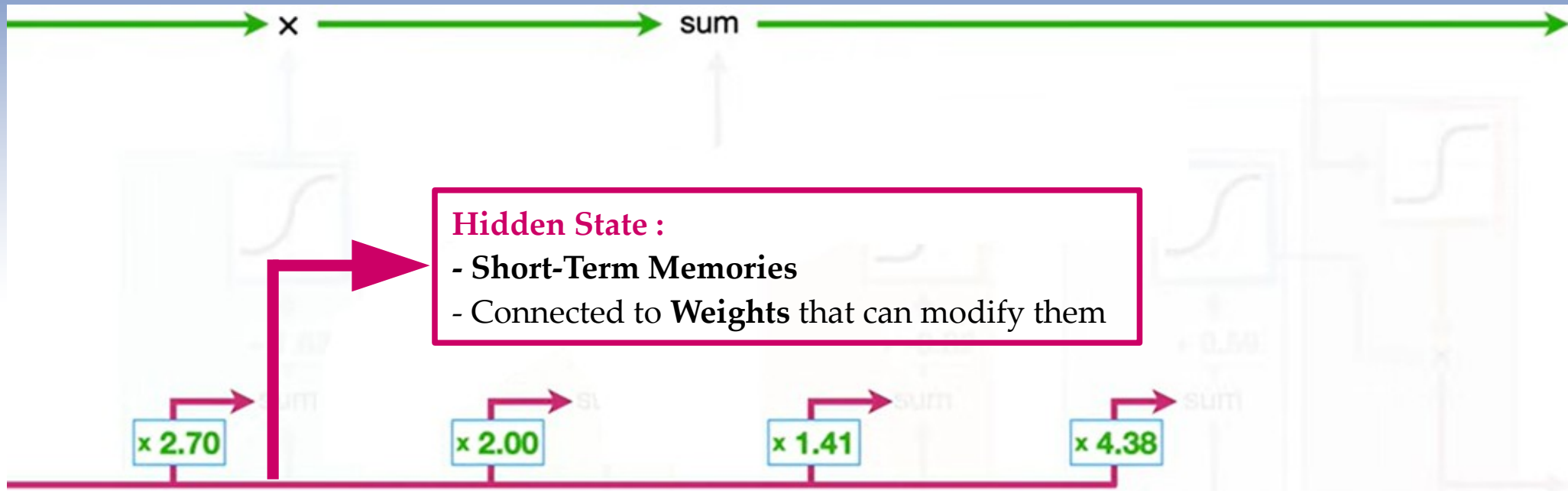
The LSTM unit

The Short-Term Memory



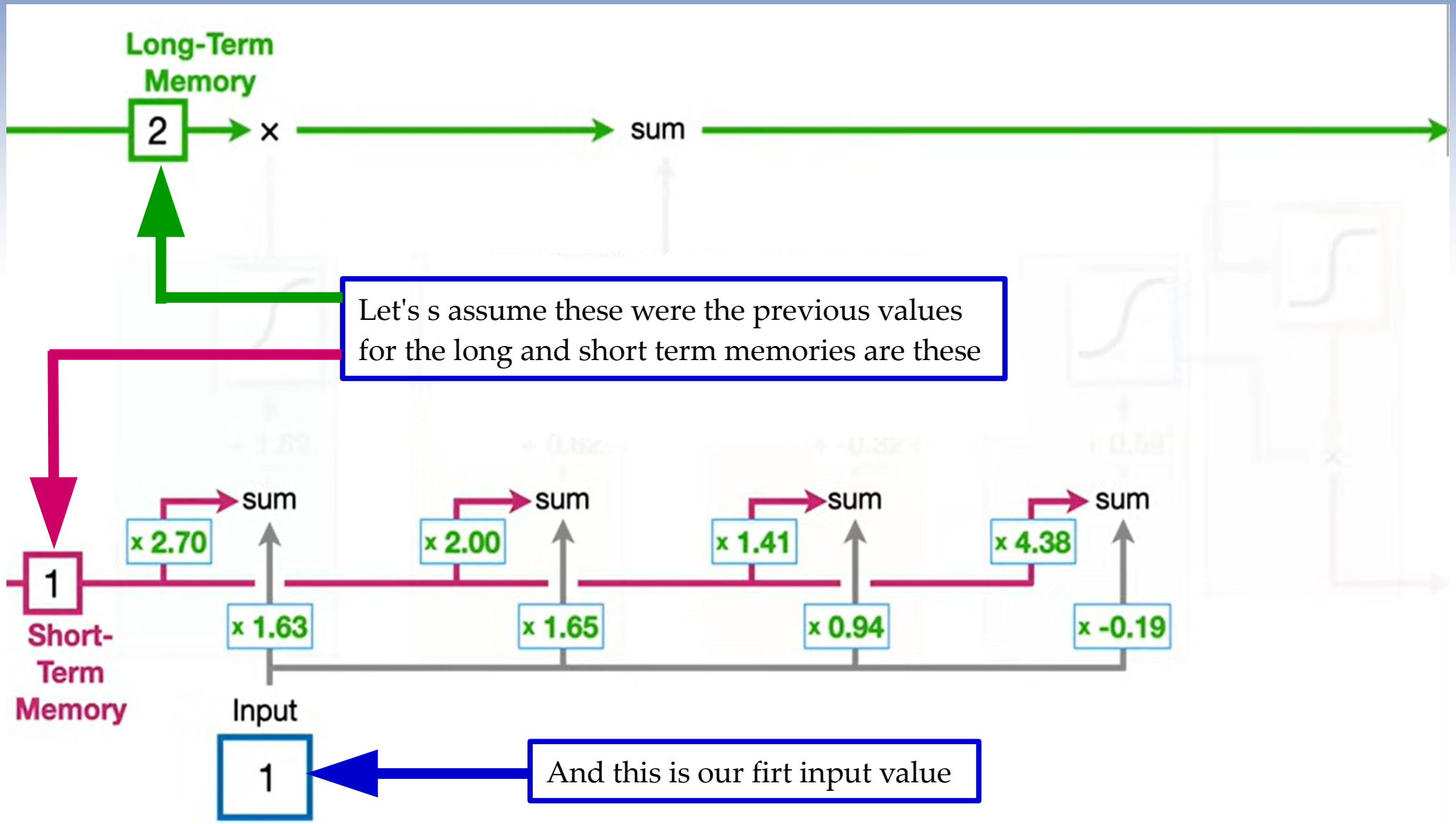
The LSTM unit

The Short-Term Memory



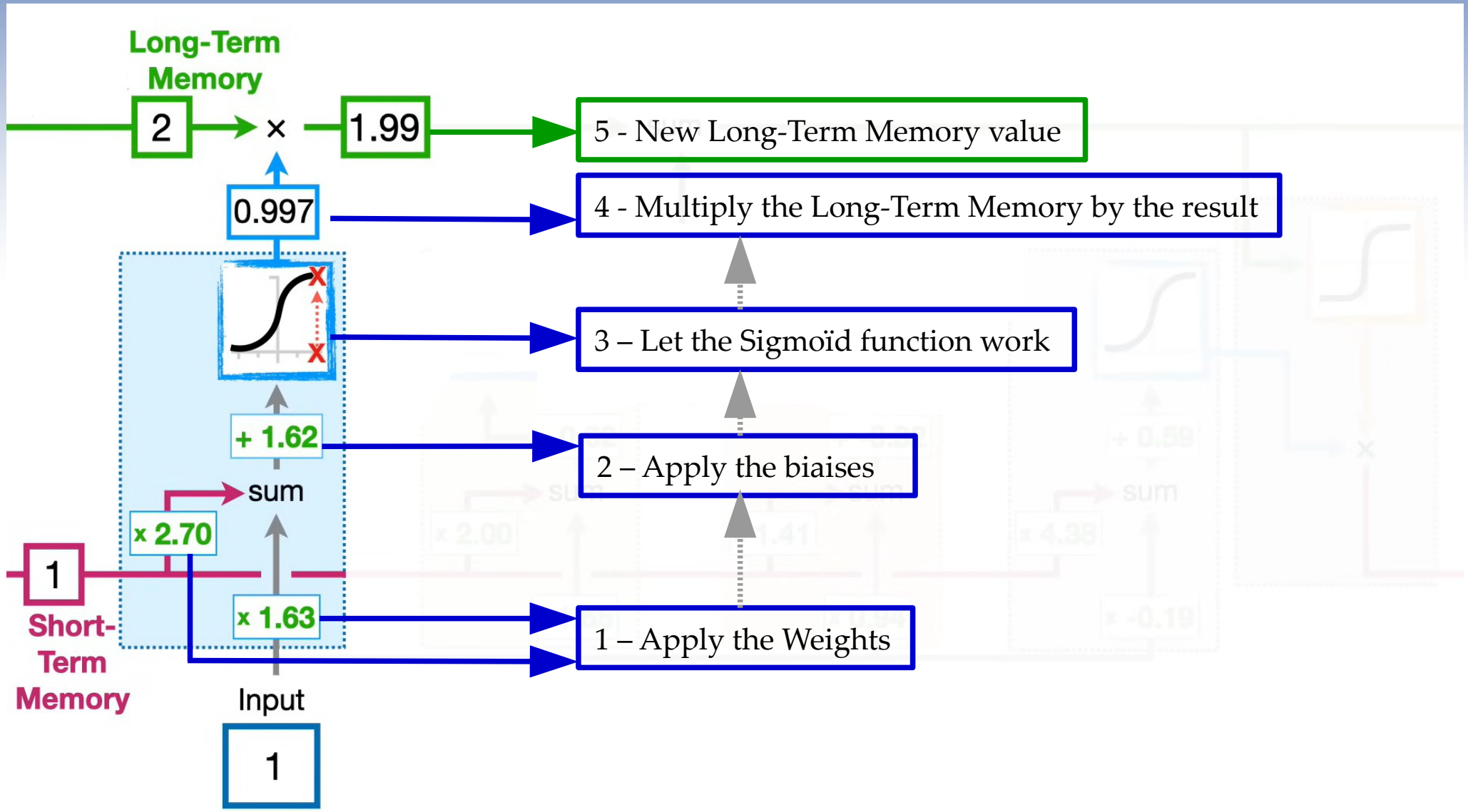
The LSTM unit

Let's start !



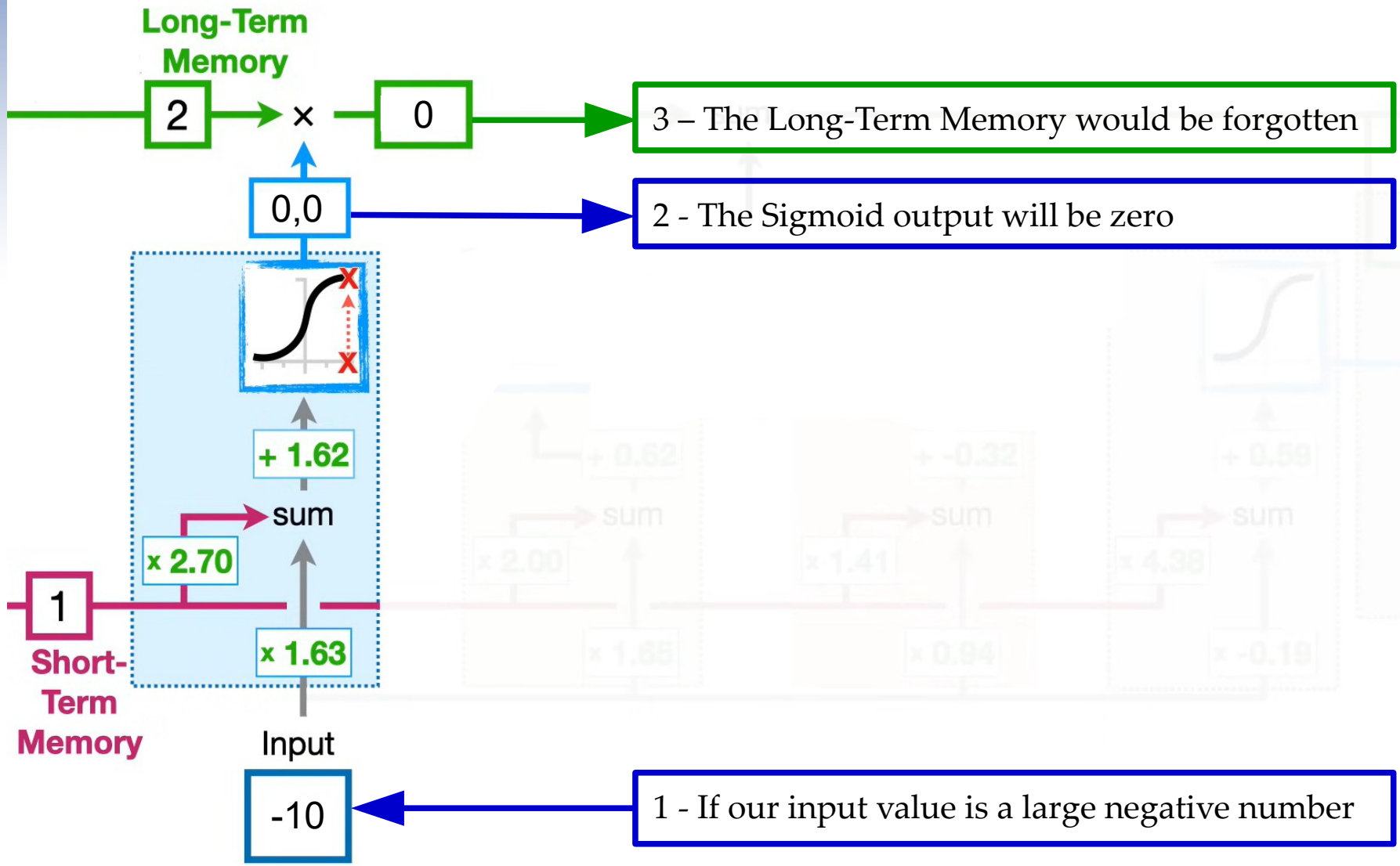
The LSTM unit

First step



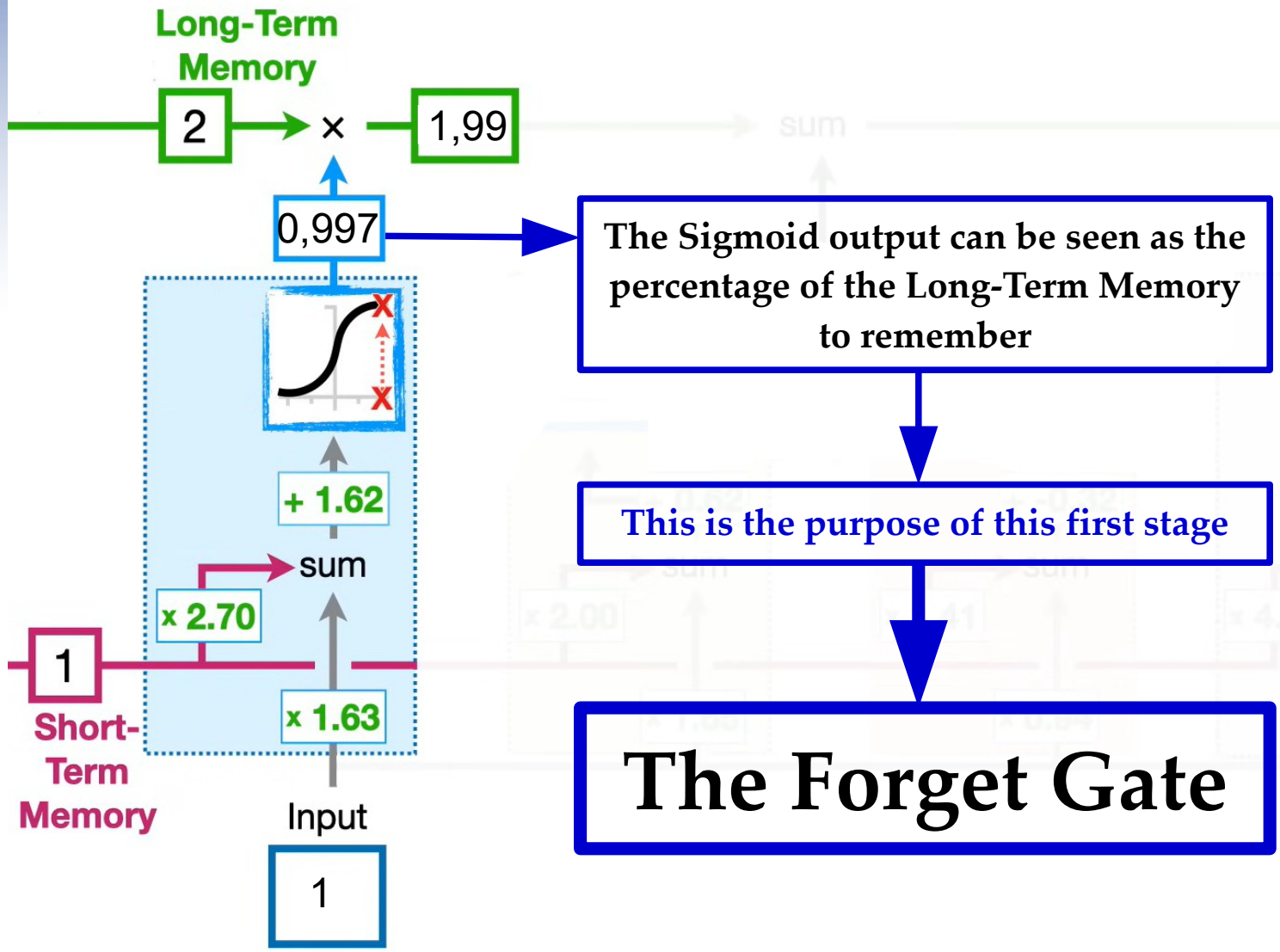
The LSTM unit

First step



The LSTM unit

First step



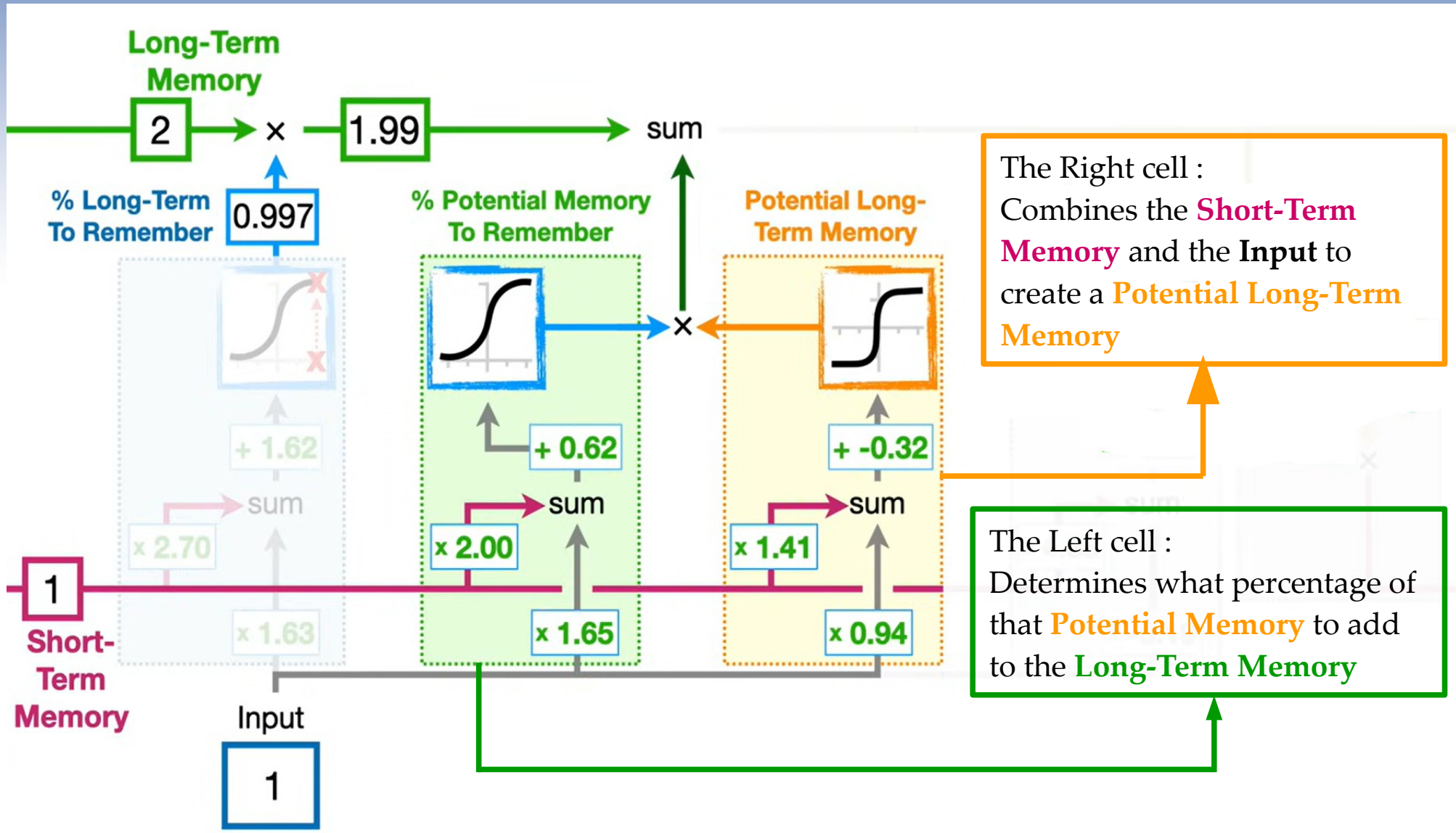
The Sigmoid output can be seen as the percentage of the Long-Term Memory to remember

This is the purpose of this first stage

The Forget Gate

The LSTM unit

Second step

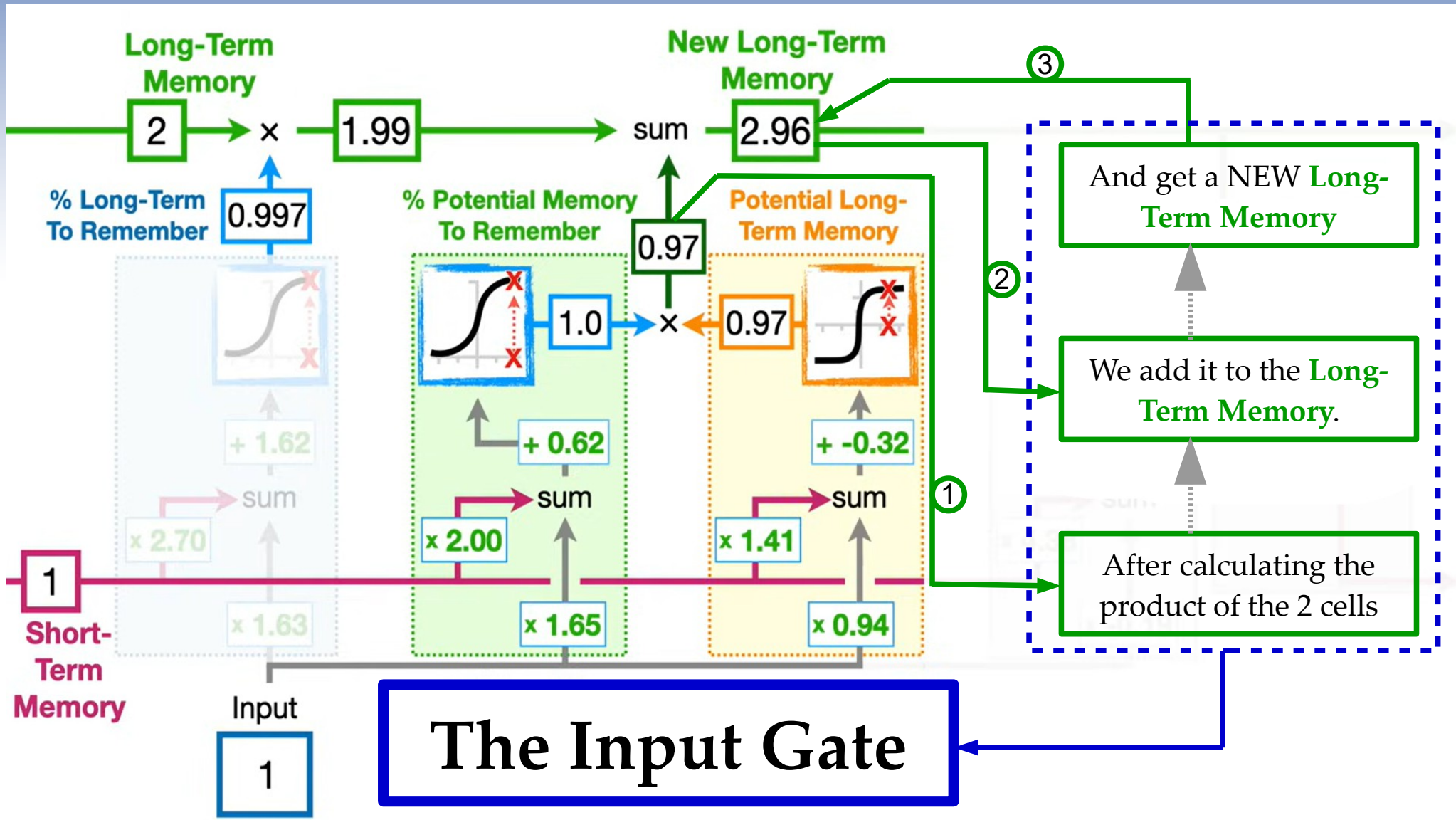


The Right cell :
Combines the **Short-Term Memory** and the **Input** to create a **Potential Long-Term Memory**

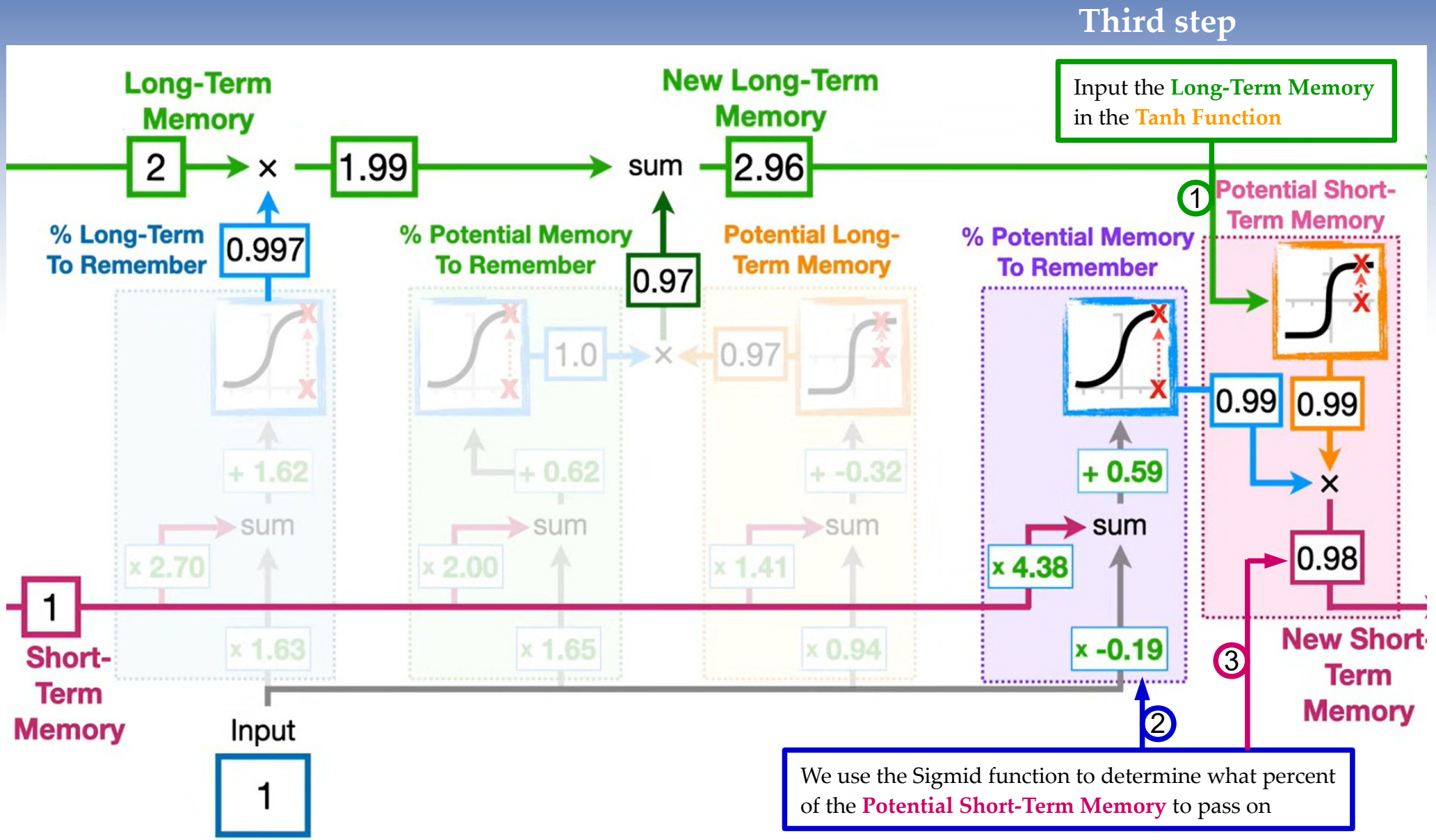
The Left cell :
Determines what percentage of that **Potential Memory** to add to the **Long-Term Memory**

The LSTM unit

Second step

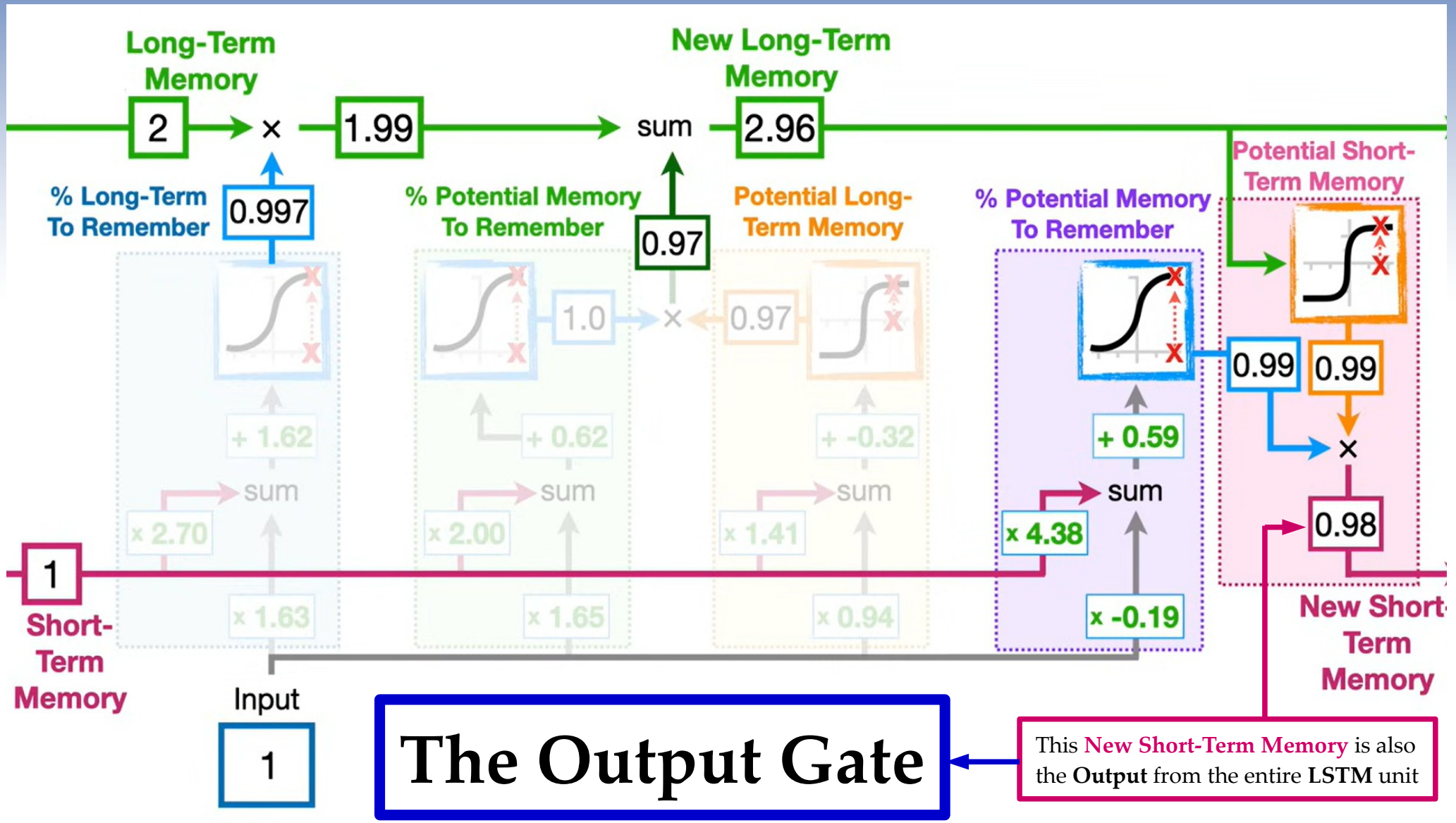


The LSTM unit



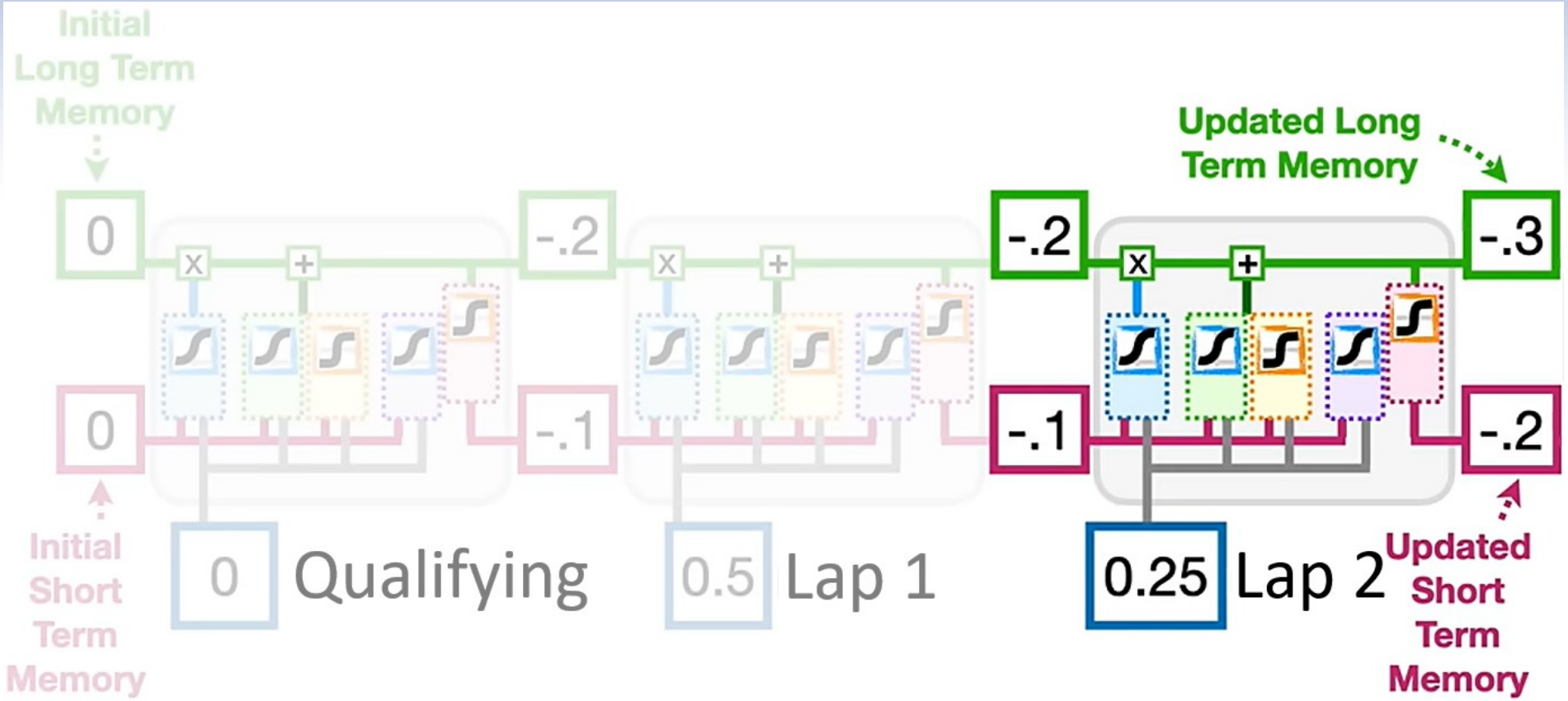
The LSTM unit

Third step



The LSTM sequence

The unit in action

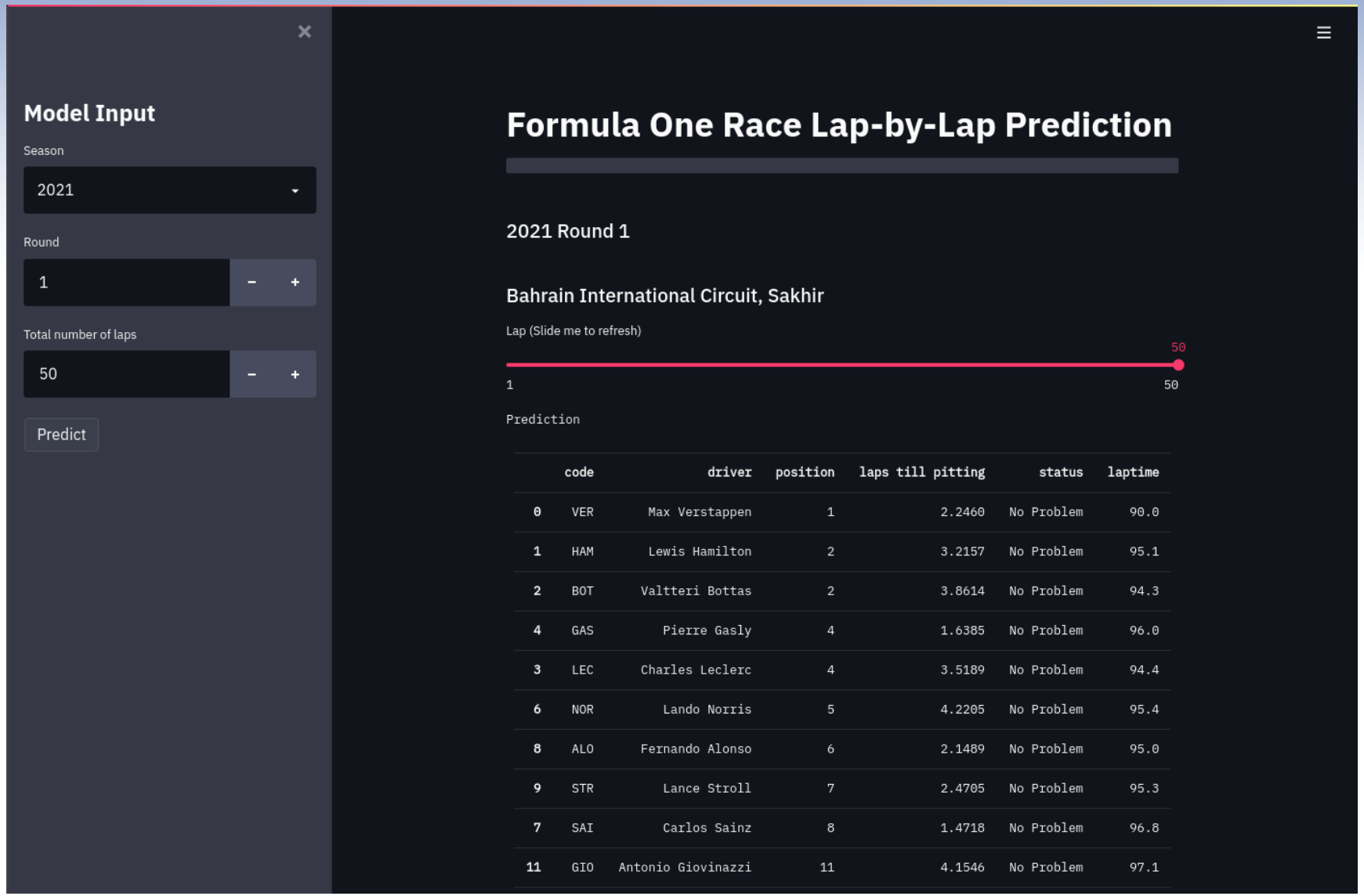


Back to the project

Raw result sample

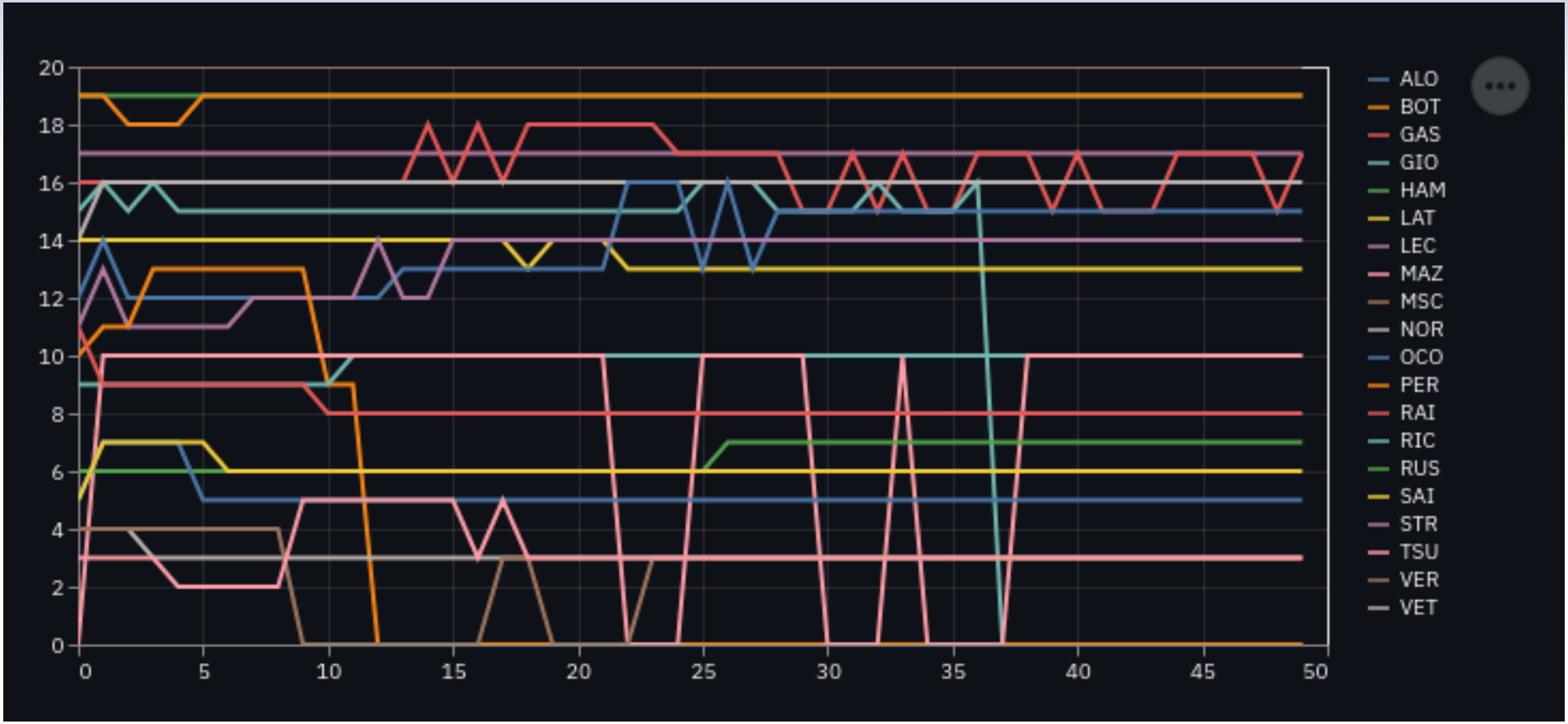
	code	driver	position	laps till pitting	status	laptime
0	HAM	Lewis Hamilton	1	0.024014	No Problem	101.5
1	VER	Max Verstappen	2	-0.023678	No Problem	101.5
2	BOT	Valtteri Bottas	3	-0.000930	No Problem	100.4
19	LAT	Nicholas Latifi	4	0.002529	No Problem	101.8
3	PER	Sergio Pérez	4	-0.001795	No Problem	101.2
4	RIC	Daniel Ricciardo	5	-0.013361	No Problem	102.3
5	SAI	Carlos Sainz	7	0.003256	No Problem	103.0
9	LEC	Charles Leclerc	8	0.022530	No Problem	102.1
12	RUS	George Russell	9	-0.024174	No Problem	103.0
8	GAS	Pierre Gasly	10	-0.025098	No Problem	103.6
7	NOR	Lando Norris	11	0.021469	No Problem	102.2
6	OCO	Esteban Ocon	12	0.019673	No Problem	101.3
16	GIO	Antonio Giovinazzi	13	0.001355	No Problem	103.6
13	VET	Sebastian Vettel	14	0.022170	No Problem	104.2
14	ALB	Alexander Albon	15	-0.034550	No Problem	103.9
15	GRO	Romain Grosjean	16	0.050112	No Problem	104.3
17	MAG	Kevin Magnussen	17	-0.007714	No Problem	103.9
11	STR	Lance Stroll	17	0.024492	No Problem	100.0
18	RAI	Kimi Räikkönen	19	0.016841	No Problem	105.6
10	KVY	Daniil Kvyat	20	0.022690	No Problem	105.5

User Interface (WIP)



	code	driver	position	laps till pitting	status	laptime
0	VER	Max Verstappen	1	2.2460	No Problem	90.0
1	HAM	Lewis Hamilton	2	3.2157	No Problem	95.1
2	BOT	Valtteri Bottas	2	3.8614	No Problem	94.3
4	GAS	Pierre Gasly	4	1.6385	No Problem	96.0
3	LEC	Charles Leclerc	4	3.5189	No Problem	94.4
6	NOR	Lando Norris	5	4.2205	No Problem	95.4
8	ALO	Fernando Alonso	6	2.1489	No Problem	95.0
9	STR	Lance Stroll	7	2.4705	No Problem	95.3
7	SAI	Carlos Sainz	8	1.4718	No Problem	96.8
11	GIO	Antonio Giovinazzi	11	4.1546	No Problem	97.1

Future result layout



Future features

→ Integrate weather, tyre compounds, track surface data.

→ Model fuel-weight impact when regulations change.

→ Explore Transformer-based architectures for improved long-term modeling.


Thank you